PolyVote

A Multi-Tenant, Ranked-Choice Election Platform with a Tamper-Evident Audit Trail

Project Review 3

Course: BACSE202 | Database Systems
Submitted by:
Tharakesharasu Thennarasu - 25BCE2699
M Pavan Durga Karthik - 25BCE2712
Shivam Malge - 25BCE0386

Faculty: Dr. Katari Balakrishna

Abstract

Campus and organisational elections are usually run on a general-purpose database that the election officer, or anyone with database access, can edit without leaving a trace. PolyVote is a database-centric election platform that addresses this at the data layer. Relational constraints and a trigger enforce one vote per voter; the voter roster and the anonymous ballots are stored so that no column links them; every ballot is sealed in a hash-chained ledger kept in a separate document store (MongoDB Atlas); and an integrity sweep reconciles ballots, receipts and voter flags to expose tampering by someone with direct storage access. Ranked-choice (instant-runoff) tallying is built in, and one schema serves many organisations. This report presents the final system, with a keyed (HMAC) ledger chain, a sealed candidate roster, an extended sweep, a standalone email one-time-password module for voter authentication, a MongoDB Atlas ledger backend and public hosting. We evaluate the system with an executable suite of 47 scenarios (attacks, abuse cases and functional tests) and repeatable benchmarks. Against a conventional hash-chain-only baseline, the number of rogue-administrator tampering scenarios detected automatically rises from 6 of 13 to 12 of 13; the remaining case, roster injection through the administrative path, is documented as a limitation. We compare the design with published verifiable-voting systems and state precisely what it does and does not claim: it improves integrity enforcement and auditability at the database layer, but it is not an end-to-end verifiable cryptographic voting system. The final build, with the email one-time code and QR-code voting, was demonstrated live in class and completed the whole voter journey successfully; screenshots of that demonstration are included as proof. A free public demo is available at https://polyvote-bacse202.onrender.com.

Keywords: electronic voting, instant-runoff voting, database integrity, hash chain, HMAC, tamper-evident logging, polyglot persistence, MongoDB, multi-tenancy, one-time password.

1. Introduction

Elections for student councils, clubs, housing societies and professional bodies are rarely run on purpose-built, audited software. They are run on spreadsheets, survey forms or small web applications backed by a single relational database. In such a setup the person who administers the database is also the person who can change the outcome, and nothing in the system can show that a result has been altered. The literature on electronic voting has concentrated on cryptographic

protocols that avoid trusting the server at all [1]-[5], and on the failures of deployed internet voting systems [6]-[9]. Between those two poles sits a large practical population of small elections for which cryptographic end-to-end verifiability is not available, yet where a much better job can be done simply by using the database properly.

PolyVote is our attempt at that middle ground. It treats the database as the place where election rules are enforced, rather than as a passive store behind application code, and it adds an independent, append-only audit trail that can be checked against the database after the fact.

1.1 Problem statement

A conventional election application suffers from three weaknesses that a database course is well placed to address:

- **Algorithmic deficiency.** Plurality counting can declare a winner who is opposed by most voters. Ranked-choice (instant-runoff) counting avoids that but is rarely implemented in small systems, and its rounds are hard to explain without a transcript.
- **Structural rigidity.** Each election is built as a one-off schema. Reusing the platform for a second organisation means duplicating it, which multiplies the administrative surface.
- **Database vulnerability.** Anyone with write access can modify, delete or insert ballots, reset a voter's "has voted" flag, or rename candidates, and a conventional application will not notice.

A fourth weakness is easy to overlook: sign-in relies on a single password, so a leaked or shared credential was enough to cast someone else's vote. The evaluators asked for a second factor, and this report responds with one.

1.2 Contributions

This report documents the final design and implementation and makes the following contributions:

- A **database-centric design** in which uniqueness, range, referential and one-vote rules are enforced by the schema and a trigger, and in which voter identity and ballot content are stored without any linking column (Sections 4.2 to 4.4).
- A **keyed, hash-chained audit ledger** stored in MongoDB Atlas, with a sealed candidate roster and a file fallback. The chain can be extended only by a holder of the application secret (Section 4.5).
- An **extended integrity sweep** that detects modified, deleted and injected ballots, renamed candidates and disagreement between voter flags, ballots and receipts (Section 4.6).
- A **standalone email one-time-password module** with its own database, hashed single-use codes, expiry, attempt limits and rate limits, kept apart from the election data (Section 4.8).
- A **reproducible evaluation**: 7 database-level prevention tests, 13 API abuse tests, 13 tampering scenarios run against both a plain hash-chain baseline and the final design, 14 OTP tests and benchmarks up to 10,000 ballots (Section 6).

- A **scoped comparative study** against published verifiable-voting systems part by part against related projects, each verdict backed by our own test results, with explicit statements of what is not claimed (Section 7).

1.3 Scope and non-goals

PolyVote targets small and medium elections whose officers are trusted to run the election but not trusted to leave no trace. It does not attempt coercion resistance, receipt-freeness against vote buyers, or protection against an adversary who controls the application host and its secrets. Those limits are discussed in Sections 3 and 9. PolyVote does not claim to compete with end-to-end verifiable systems on cryptographic strength; its claim is that, for the class of elections described, it removes the easiest and most likely attacks and makes the remaining ones detectable.

1.4 Organisation

Section 2 reviews related work. Section 3 states the threat model and goals. Section 4 describes the design and Section 5 the implementation. Section 6 reports the evaluation, Section 7 the comparative study, Section 8 the final implementation status, Section 9 limitations and future work, and Section 10 concludes.

2. Background and Related Work

2.1 Verifiable electronic voting

Helios introduced web-based open-audit voting, in which ballots are encrypted in the browser and anyone can verify the published tally [1]. Belenios builds on a similar idea with stronger guarantees on eligibility and privacy [2], and Civitas adds coercion resistance at the cost of considerably more complex procedures [3]. Prêt à Voter [4] and Scantegrity II [5] aim at voter-verifiable paper-based elections, using randomised candidate orderings and confirmation codes respectively. These systems rest on cryptographic protocols and, in most cases, on trusted setup such as key ceremonies and independent trustees. They provide properties PolyVote does not: a voter can check that their ballot was recorded as intended without trusting the server. They also carry a procedural and operational burden that small organisations seldom accept.

2.2 Deployed internet voting

Security analyses of deployed systems are sobering. The Estonian internet voting system, widely regarded as the most mature, was found to have significant weaknesses in its operational and procedural security [6]. Mobile and web voting products used in United States elections have been shown to contain vulnerabilities that could expose or alter votes [7], [8]. A National Academies study concluded that internet voting could not currently be secured [9], and Park et al. argue that blockchain-based voting, often proposed as a remedy, combines the problems of internet voting with new ones [14]. We take two lessons from this body of work. First, a design must state its threat model explicitly rather than advertise a technology. Second, claims about "immutable" ledgers must be examined: a ledger is only as trustworthy as the party who can write to it.

2.3 Tamper-evident logging and ledger databases

Hash chaining for time-stamping was introduced by Haber and Stornetta [10], and Schneier and Kelsey showed how to build secure audit logs in which an attacker who compromises the logging

machine cannot undetectably alter earlier entries [11]. Crosby and Wallach later gave efficient data structures for tamper-evident logging with logarithmic proofs [12], and Certificate Transparency applies Merkle trees to public logs [13]. In the database community, Snodgrass et al. studied tamper detection in audit logs [28] and Pavlou and Snodgrass the forensic analysis of database tampering [27]. Commercial and research ledger databases, such as SQL Ledger in Azure SQL Database, add cryptographically verifiable history to a relational engine [26]. PolyVote uses the simplest member of this family, a linear hash chain, and adds a keyed variant so that an attacker with access to the stored chain but not to the key cannot rebuild it.

2.4 Ranked-choice voting and its auditing

Instant-runoff voting (IRV) eliminates the weakest candidate round by round until one holds a majority of continuing ballots. Its outcome can hinge on a small number of ballots, and computing how many ballots would have to change to alter it is a research problem in its own right [15], [17]. Risk-limiting audits for IRV have been developed [16], [18]. PolyVote does not implement a statistical audit. It provides a deterministic tally with a round-by-round transcript, and it flags the one source of arbitrariness in the algorithm, ties for last place, in the interface.

2.5 Multi-tenancy and polyglot persistence

Aulbach et al. compare schema-mapping techniques for multi-tenant databases and show that a shared schema with a tenant identifier is the most economical approach [19], [20]. PolyVote follows that approach: one schema, with the organisation and election identifiers partitioning all data. Polyglot persistence, the use of different storage technologies for different kinds of data [21], [22], is a well-known response to the observation that no single data model fits all needs [23], [24]. PolyVote uses a relational store for rule-heavy transactional data and a document store for the append-only audit log.

2.6 Database security and integrity enforcement

Bertino and Sandhu survey database security and list integrity constraints, access control and auditing as the foundations [25]. Declarative constraints and triggers are the standard mechanisms for enforcing integrity inside the engine [29], [30]. Their value for voting is that they apply to every client, including a database administrator's console, provided the engine is configured to enforce them. Section 6.2 shows one case where it is not, and discusses the consequence.

2.7 Authentication factors

NIST SP 800-63B treats a memorised secret as one factor and requires a separate factor, such as a one-time code delivered to an authenticator the claimant controls, for higher assurance [31]. HOTP and TOTP define counter- and time-based one-time passwords [32], [33]. Email as an authentication channel is convenient but only as strong as the mailbox [37], and SMS or mobile channels have documented weaknesses [36]. Guidance on password storage recommends salted, slow key-derivation functions [34], [35]. PolyVote adds an emailed code as a second factor, and its passwords are stored with plain SHA-256, which does not follow that guidance (Section 9).

2.8 Receipts and ballot secrecy

Voter receipts that prove a vote was counted without revealing its content were studied by Chaum [38]. Benaloh and Tuinstra showed that receipts must not allow a voter to prove how they voted if vote selling is to be resisted [39], and Juels et al. formalised coercion resistance [40]. PolyVote's

receipt is deliberately weak in this respect: it proves that a ballot is recorded and still counted, but does not allow a voter to verify the ballot's content, and a voter who reveals the token reveals nothing about the ranks without database access. It is neither receipt-free in the strong sense nor verifiable end to end.

2.9 Positioning

The systems above optimise for cryptographic assurance. The database literature offers tamper-evident structures but rarely applies them to a complete, usable election workflow. PolyVote occupies the gap: a complete multi-tenant ranked-choice application whose integrity story is built from database features, whose threats are enumerated, and whose claims are tested by an executable suite.

3. Threat Model and Security Goals

3.1 Assets

The assets to protect are (i) the correctness of the tally, (ii) the secrecy of each voter's ranking, (iii) the one-person-one-vote property, (iv) the eligibility of those who vote, and (v) the ability of an independent party to detect that any of these has been violated.

3.2 Adversaries

We consider the adversaries in Table 1. They are ordered by capability. The first is addressed by authentication, the middle ones by the database and ledger design, and the last two mark the boundary of what the design claims.

ID	Adversary	Capability	Treatment
A1	Voter with a leaked or guessed password	Can sign in as someone else	Email one-time code; one-vote trigger
A2	Network observer	Reads or alters traffic	Out of scope beyond TLS at the tunnel
A3	Rogue database administrator	Read/write on the election database only	Integrity sweep detects
A4	Ledger-store administrator	Read/write on the ledger store only	Hash chain with HMAC detects
A5	Insider with both stores, no secrets	Edits both stores consistently	Keyed chain and sweep detect
A6	Insider with both stores and the key	Can forge valid blocks	Not covered; needs external anchoring
A7	Roster injection via the admin path	Adds a ghost voter, votes as them	Not covered; needs roster commitment

Table 1: Adversary model and how each adversary is treated.

ID	Adversary	Capability	PolyVote response	Status
A1	Voter with a leaked or guessed password	Signs in as someone else	Emailed one-time code; one-vote trigger	Prevented
A2	Network observer	Reads or alters traffic	Out of scope beyond TLS at the tunnel	Not covered
A3	Rogue database administrator	Read/write on the election database only	Integrity sweep: modified, deleted, injected, mismatch	Detected
A4	Ledger-store administrator	Read/write on the ledger store only	HMAC hash chain; unique block index	Detected
A5	Insider with both stores, no secrets	Edits both stores to match each other	Keyed chain + sweep catch forged receipts, rewrites	Detected
A6	Insider with both stores and the ledger key	Forges valid blocks	Needs external anchoring (future work)	Not covered
A7	Roster injection through the admin path	Adds a ghost voter, votes as them through the API	Needs a committed roster (future work)	Not covered

Figure 1: Threat model overview: adversaries, capabilities and treatment.

In plain words. Think of this table as a list of people who might try to cheat. Green means PolyVote stops them. Blue means PolyVote cannot always stop them but will catch them afterwards. Red means PolyVote does not protect against them yet. We show the red rows on purpose, so nobody is surprised later.

3.3 Security goals

Goal	Statement	Mechanism
G1	No voter can vote twice	Trigger, `has_voted` CHECK, per-election UNIQUE roster, single-use sessions, write lock
G2	Only rostered, verified voters vote	Roster lookup, password, emailed one-time code
G3	Ballots are anonymous in storage	No column links `BALLOT_RANKS` to `VOTER_PARTICIPATION`; random ballot tokens
G4	Malformed ballots are rejected	CHECK, composite PK, FK, API validation
G5	Modification, deletion or insertion of stored ballots is detectable	Per-ballot ledger receipt, integrity sweep
G6	The ledger cannot be silently rewritten by someone without the key	HMAC chain, unique block index, store separate from the database
G7	Candidate attribution cannot be changed after voting	Roster hash sealed in each block
G8	A voter can confirm their ballot is recorded and counted	Receipt token and public receipt check

Table 2: Security goals and the mechanisms that serve them.

3.4 Assumptions and non-goals

- The application host and its environment secrets (`MONGO_URI`, `LEDGER_KEY`, `SMTP_PASS`) are not controlled by the adversary, except for A6.
- A voter's mailbox is under that voter's control. Email is a weaker factor than a hardware token, and we say so.
- The election officer chooses the roster honestly at registration. Verifying roster provenance is future work (A7).
- We do not address coercion, vote selling, denial of service, or compromise of the voter's own device.

4. System Design

4.1 Architecture

PolyVote is a single Python service built on FastAPI that exposes a REST interface and serves the React front end. Behind it are three stores, deliberately separated by what each is trusted with (Figure 2). The election database is a relational SQLite file that holds the roster, candidates and anonymous ballots. The verification database is a second SQLite file that holds voter email addresses and one-time-code challenges. The audit ledger is a MongoDB Atlas collection, with a local JSON file used when no connection string is configured. Secrets (database connection string, ledger key, SMTP credentials) live in environment variables, outside every store.

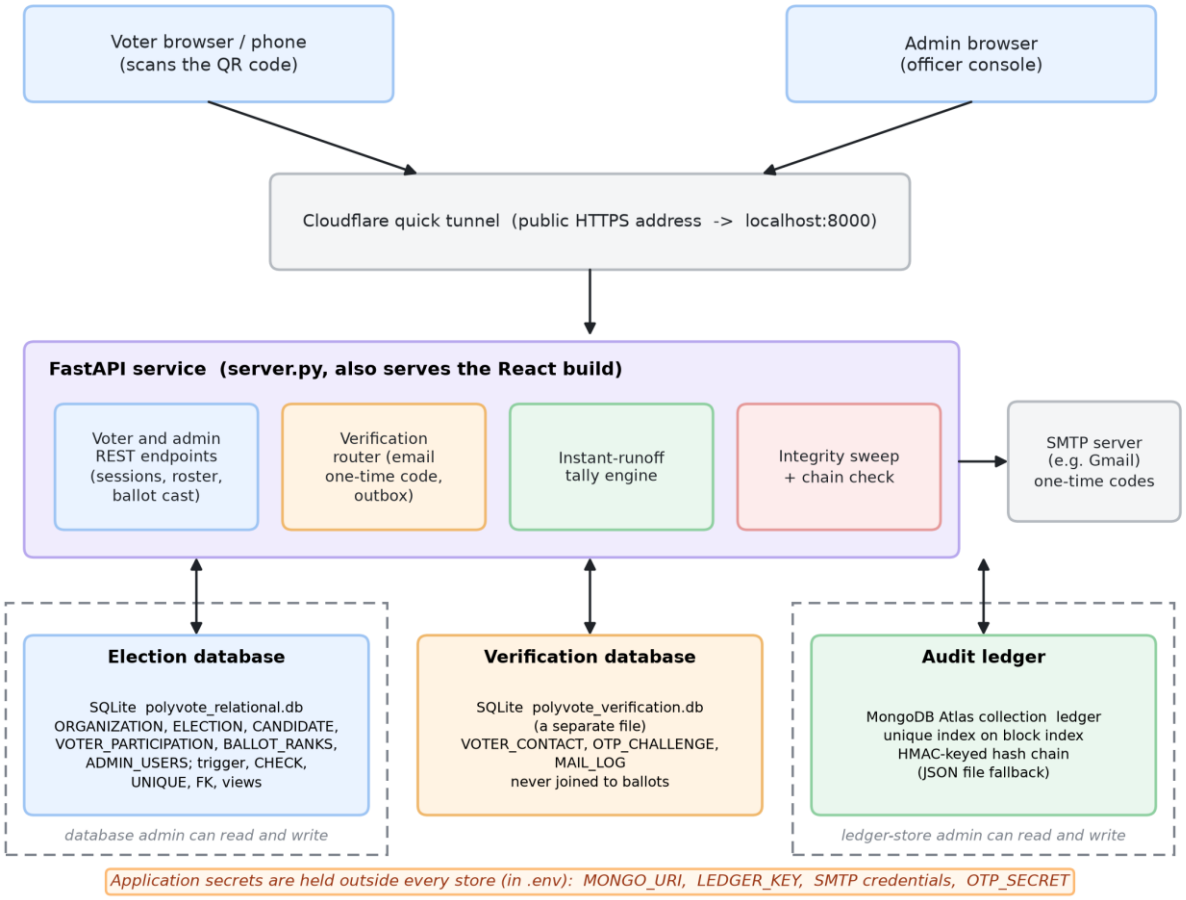


Figure 2: Architecture of the final system with trust boundaries.

In plain words. Read the picture from top to bottom. A voter's phone or the officer's browser talks to one program, the FastAPI service. That program keeps its information in three separate places: the ballots in one database, the voters' email addresses and one-time codes in a second database, and the audit chain in MongoDB. They are kept apart so that someone who can open one of them cannot read or change the others. The secrets (database password, the key for the chain, the email password) are stored in a separate settings file that is not part of any database.

The separation matters for the threat model. An administrator of the election database cannot touch the ledger or the verification data, an administrator of the ledger store cannot touch ballots, and neither holds the secrets. Only the application host sees all three, which is why compromise of the host is outside the model (A6).

4.2 Data model

The election schema has six tables (Figure 3). `ORGANIZATION` hosts `ELECTION` rows, which own `CANDIDATE`, `VOTER_PARTICIPATION` and `BALLOT_RANKS` rows. `ADMIN_USERS` is a stand-alone table for officer sign-in. Multi-tenancy follows the shared-schema pattern [19]: all tenant data hangs off the election, and every query is scoped by `election_id`.

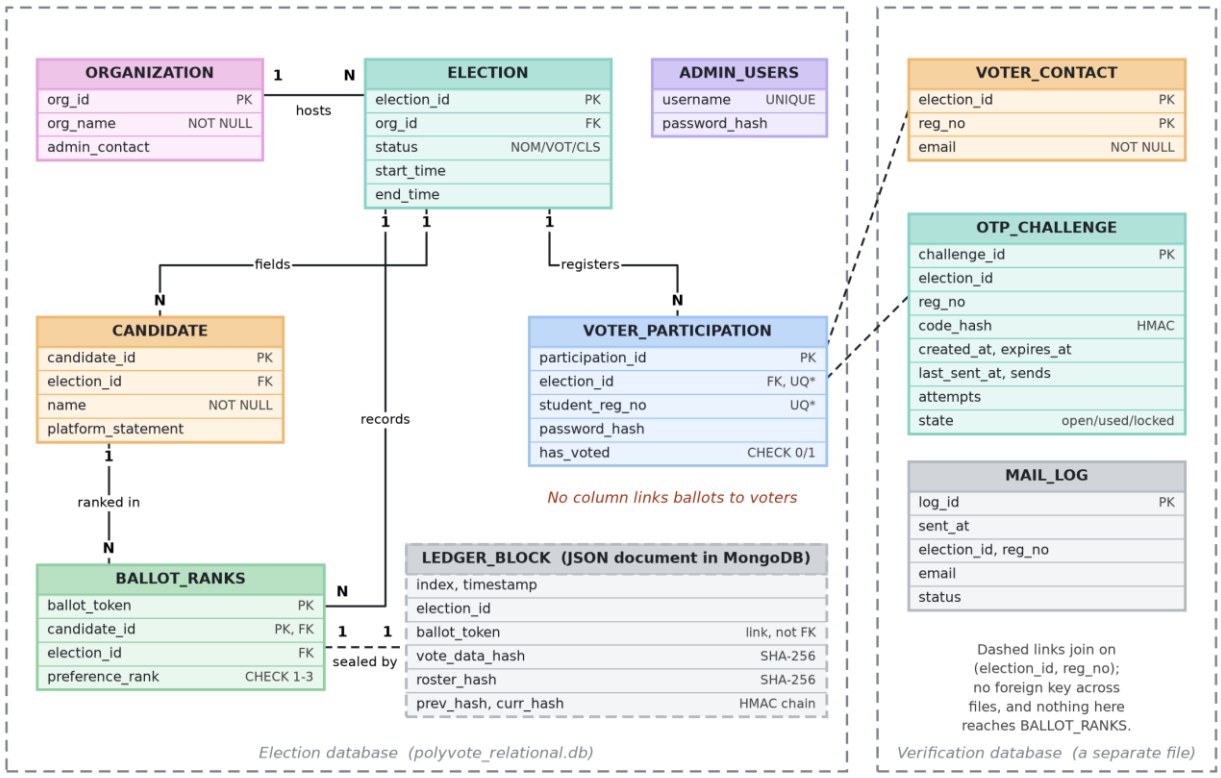


Figure 3: Entity-relationship diagram: election database, verification database and ledger block.

In plain words. Each box is a table and each line shows how two tables are linked. The important point is what is missing: the ballot table has no line to the voter table, so nobody can look up who voted for whom. The dashed box is one block of the audit chain, and the right-hand frame is the separate database for emails and codes.

The rules the schema enforces are listed in Table 3. They are the first line of defence and apply to any client that speaks SQL, including an administrator console, provided foreign-key enforcement is switched on for the connection (Section 6.2).

Rule	Mechanism	Where
A voter cannot be marked as voted twice	`BEFORE UPDATE OF has_voted` trigger raises ABORT when old and new are both 1	`VOTER_PARTICIPATION`
`has_voted` is 0 or 1	CHECK	`VOTER_PARTICIPATION`
One roster entry per voter per election	UNIQUE (`student_reg_no`, `election_id`)	`VOTER_PARTICIPATION`
A ballot ranks 1 to 3 choices	CHECK (`preference_rank` BETWEEN 1 AND 3)	`BALLOT_RANKS`
A candidate appears once per ballot	Composite PK (`ballot_token`, `candidate_id`)	`BALLOT_RANKS`
Ballots reference real candidates and elections	Foreign keys	`BALLOT_RANKS`, `CANDIDATE`, `ELECTION`

Rule	Mechanism	Where
One verification record per voter contact	PK (`election_id`, `reg_no`)	`VOTER_CONTACT`
A challenge is open, used or locked	CHECK on `state`	`OTP_CHALLENGE`

Table 3: Integrity rules enforced inside the database engines.

4.3 Separating the voter from the ballot

A ballot is stored under a random UUID, the ballot token, and the roster stores only whether a voter has voted. There is no column in `BALLOT_RANKS` that appears in `VOTER_PARTICIPATION`, other than the election identifier, and no timestamp on the roster row that could be matched to the order of ballots. A database administrator reading both tables can see that 45 voters voted and that 45 ballots exist, but not which ballot belongs to whom. The voter's email address is held in a different database again, keyed by registration number, so it cannot be joined to a ballot even if the election database is compromised. Table 12 reports the automated check of this property.

4.4 Casting a ballot

Casting is the only operation that touches all the structures, so it is serialised and transactional (Figure 4). After authentication the server takes a process-wide write lock, re-checks the election status, validates the ranking, and verifies that the voter has not already voted. In a single database transaction it sets `has_voted`, inserts the ranked rows under a new random token, and commits. It then computes the candidate roster hash, appends a sealed block to the ledger, invalidates the session and returns the token to the voter.

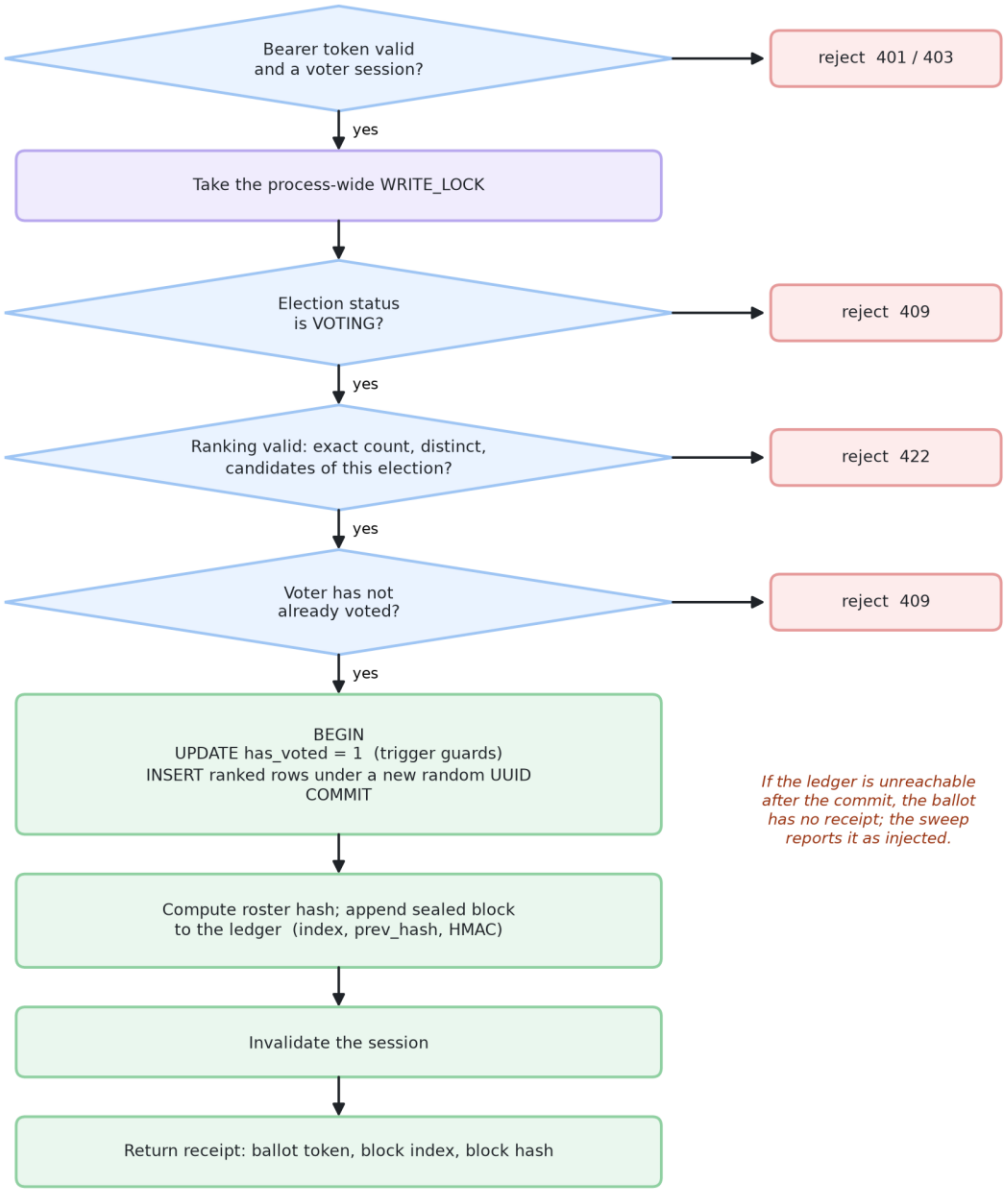


Figure 4: Flow of a ballot cast, with the failure exits.

In plain words. Follow the arrows downward. Each diamond is a check, and if a check fails the vote is rejected (the red boxes on the right). Only a ballot that passes every check is saved, sealed in the audit chain and given a receipt.

The ledger append happens after the database commit. If the ledger were unreachable at that moment the ballot would exist without a receipt. The extended sweep detects exactly that situation (Section 4.6), and the receipt-lookup page shows a voter whether their token is present. A two-phase scheme that writes the receipt first would trade this for orphan receipts; we chose the variant that fails visibly.

4.5 The audit ledger

Each ballot produces one block: its position in the chain (`index`), a timestamp, the election, the ballot token, `vote_data_hash`, `roster_hash`, the previous block's hash (`prev_hash`) and the block's own hash (`curr_hash`). `vote_data_hash` is the SHA-256 of the ordered (candidate, rank) pairs. `roster_hash` is the SHA-256 of the election's (candidate id, name) pairs at the time of the vote. `curr_hash` is computed over all other fields.

A conventional chain computes `curr_hash` with plain SHA-256 and no key. That makes the chain self-consistent but not authenticated: anyone who can write the ledger store can change a ballot's hash and then recompute every later block. The final design therefore computes `curr_hash` as HMAC-SHA-256 under a secret key (`LEDGER_KEY`) held outside the ledger and database. Verification recomputes each block with the same key and compares with a constant-time comparison. Blocks written before the key was introduced remain valid because the first `LEDGER_LEGACY_UNTIL` blocks are verified with plain SHA-256, which keeps old elections readable.

Two storage backends sit behind the same interface. With `MONGO_URI` set, blocks are documents in a MongoDB collection with a unique index on `index`. The unique index serves as a compare-and-swap: two concurrent appenders that compute the same next index cannot both insert, so the chain cannot fork. Without a connection string, blocks go to a local JSON file, protected by an in-process lock. The backend is chosen once at start-up and reported by `/api/health`.

4.6 The integrity sweep

The sweep recomputes the state of an election from the database and compares it with the ledger. The first two rules are the basic checks; the other three close gaps the evaluation exposed in a basic sweep (Table 4).

Rule	Detects	How
1. Modified	A ballot whose ranks changed after sealing	Recompute SHA-256 of the ballot's ranks; compare with the block's `vote_data_hash`
2. Deleted	A sealed ballot missing from the database	A block exists for a token that has no rows in `BALLOT_RANKS`
3. Injected (new)	A ballot with no receipt	Rows exist in `BALLOT_RANKS` for a token with no block
4. Roster changed (new)	Candidate names altered after voting	Current roster hash differs from the hash sealed in the blocks
5. Count mismatch (new)	Flags, ballots and receipts disagree	Compare voted flags, distinct ballot tokens and blocks for the election

Table 4: Rules applied by the integrity sweep.

In addition, the chain check verifies every block's link and hash across the whole ledger. The two checks are complementary: the chain check finds edits inside the ledger, and the sweep finds edits to the database relative to a trusted ledger.

4.7 Instant-runoff tally

The tally loads ballots as ordered lists of candidates and repeats three steps: count first preferences among continuing candidates, declare a winner if one holds more than half of the continuing ballots, otherwise eliminate the candidate with the fewest and transfer their ballots to the next surviving preference. Ballots that run out of preferences are exhausted. The engine returns a transcript of every round. Ties for last place are broken by candidate order, which is arbitrary; the interface marks any round where that happens so it is not mistaken for a principled result.

4.8 Email one-time-password verification

A password alone no longer opens a ballot. When the OTP setting is enabled, a correct password produces a challenge instead of a session (Figure 5). The server generates a six-digit code with a cryptographically secure generator, stores only an HMAC of it, and emails the plain code from a background thread so the page never waits on the mail server. The voter enters the code, and only then does the server issue the voting session.

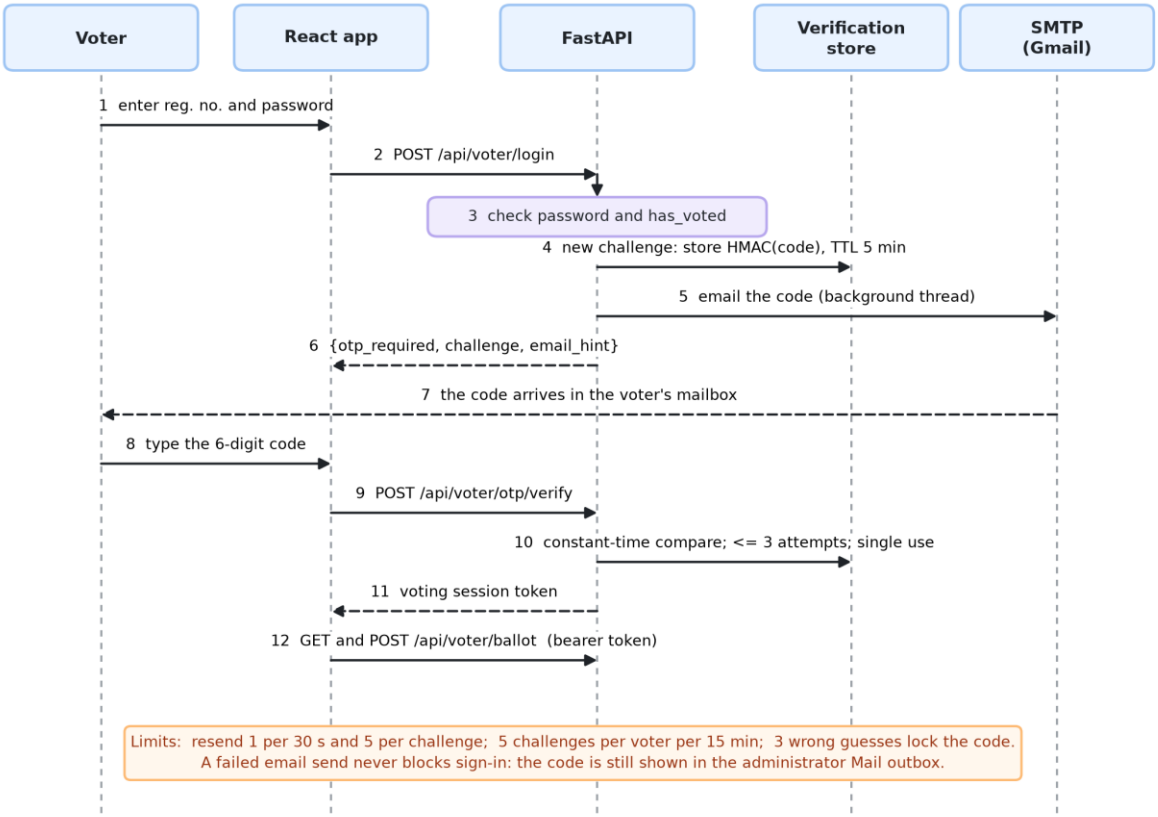


Figure 5: Sign-in with a one-time code: sequence of messages.

In plain words. Time runs from top to bottom. The voter types a password, but the server does not let them in yet. It emails a 6-digit code and waits. Only when the voter types the right code does the server hand over the ballot. A stolen password alone is therefore not enough.

Parameter	Value	Purpose
Code	6 digits, `secrets.randbelow`	Unpredictable; zero-padded

Parameter	Value	Purpose
Storage	HMAC-SHA-256 of `challenge:code` under a server secret	The database never holds a usable code
Validity	5 minutes	Limits the window for theft or guessing
Guesses	3 per code, then locked	About 3 in 10^6 chance per code
Resends	1 per 30 s, at most 5 per challenge	Resend cannot be used to keep guessing
Challenges	5 per voter per 15 minutes	Slows repeated password-then-guess attempts
Use	Single use	A captured code cannot be replayed
Delivery	SMTP over STARTTLS; failure logged, never blocks sign-in	The admin outbox still shows the code

Table 5: Parameters of the one-time-code mechanism.

Voter addresses are entered per voter, or generated in bulk from one base mailbox using plus-addressing (`you+25bce0001@domain`), which lets a single inbox receive every code during a demonstration. Contacts, challenges and the mail log live in `polyvote_verification.db`, a different file from the election database. An administrator page, the Mail outbox, lists every code issued; plaintext codes are kept in memory only and are hidden after ten minutes.

4.9 Election lifecycle and multi-tenancy

An election moves through Nomination, Voting and Closed, one step at a time and never backwards. Candidates may change only in Nomination, the roster may grow in Nomination and Voting, and ballots are accepted only in Voting. Because the lifecycle is stored in the database and enforced by the API, a closed election is read-only for voters. Many organisations share the platform; each election belongs to one organisation, and administrators see all of them, which is the model for a campus election office.

4.10 Deployment and operation

The service runs from one command (`python -m uvicorn server:app --port 8000`) and serves the compiled front end from the same port. For the review it is exposed to the internet through a Cloudflare quick tunnel, which gives a temporary public HTTPS address without opening ports; the voter page shows a QR code for that address so a phone can open the ballot. The ledger is hosted on a free MongoDB Atlas cluster. All configuration is read from a git-ignored `.env` file (Table 6). We describe this honestly as public hosting from a laptop rather than cloud deployment: the tunnel address changes when it restarts, and the host must stay on.

For a link that does not depend on the laptop, the repository also builds as a Docker image (`Dockerfile`) and includes a Render blueprint (`render.yaml`) for a free public demo at `https://polyvote-bacse202.onrender.com`. On a free plan the service sleeps after about 15 minutes idle (the next visit takes about a minute) and its disk is wiped on every restart, so the demo seeds a fresh election at start-up. A public demo has no mailbox the visitor can open, so in demo mode the one-time code is shown on the screen, a banner says so, and factory reset is disabled. The `/demo` page lists every page of the application with a link and a one-line description, and gives a three-minute tour. A second blueprint (`render-live.example.yaml`) sends real emailed codes through an SMTP relay on port 2525, because the free plan blocks the usual mail ports.

Variable	Effect
MONGO_URI	Use MongoDB Atlas for the ledger (file fallback if unset or unreachable)
LEDGER_KEY, LEDGER_LEGACY_UNTIL	Enable the keyed (HMAC) chain; number of earlier blocks that use plain SHA-256
OTP_REQUIRED	Require the emailed code at voter sign-in
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS	Send real email; otherwise codes appear only in the admin outbox
PUBLIC_URL	Address encoded in the voter-page QR code when opened on localhost
ADMIN_PASSWORD	Replace the default administrator password

Table 6: Configuration read from the environment.

5. Implementation

5.1 Technology and code size

The back end is Python 3.12 with FastAPI, SQLite through the standard library, pandas for the tally and pymongo for the ledger. The front end is React 19 with TypeScript, built by Vite. The Streamlit interface from Review 1 is retained for comparison. Table 7 lists the size of each part.

Part	Lines
Backend (Python)	1,484
Frontend (TypeScript/TSX)	3,000
Styles (CSS)	473
Experiments (Python)	704
Total	5,661

Table 7: Source size (non-generated, including blank lines).

5.2 Selected listings

The duplicate-vote trigger is the first line of defence for G1:

```
CREATE TRIGGER trg_block_duplicate_vote
    BEFORE UPDATE OF has_voted ON VOTER_PARTICIPATION
    FOR EACH ROW
    WHEN OLD.has_voted = 1 AND NEW.has_voted = 1
    BEGIN
        SELECT RAISE(ABORT, 'DBMS Trigger Violation: Voter has already submitted a ballot.');
    END;
```

The ledger keys its chain with an HMAC when a key is configured, and falls back to plain SHA-256 for legacy blocks:

```
def _block_hash(index, block):
    data = json.dumps(block, sort_keys=True).encode()
    if LEDGER_KEY and index > LEGACY_UNTIL:
        return hmac.new(LEDGER_KEY, data, hashlib.sha256).hexdigest()
    return hashlib.sha256(data).hexdigest()
```

Appending to the MongoDB ledger relies on the unique index on the block position, retrying if another writer won the slot:

```
for _ in range(5):
    last = _coll.find_one({}, {'_id': 0}, sort=[('index', -1)])
    block = _seal(election_id, ballot_token, ranked_payload,
                  (last['index'] + 1) if last else 1,
                  last['curr_hash'] if last else GENESIS, roster_hash)
    try:
        _coll.insert_one(dict(block))
        return block
    except DuplicateKeyError:
        continue
```

The sweep's three additions are short, because the data they need is already in hand:

```
report["injected"] = [t for t in grouped if t not in ledger_dict]
sealed = {b['roster_hash'] for b in blocks if b.get('roster_hash')}
report["roster_changed"] = bool(sealed) and sealed != {current_roster}
counts = {"voted_flags": int(voted_flags), "ballots": len(grouped), "receipts": len(ledger_dict)}
if len(set(counts.values())) > 1:
    report["count_mismatch"] = counts
```

Verifying a one-time code compares in constant time and records every failed attempt:

```
if not hmac.compare_digest(row['code_hash'], _hash(challenge_id, code)):
    attempts = row['attempts'] + 1
    locked = attempts >= MAX_ATTEMPTS
    c.execute('UPDATE OTP_CHALLENGE SET attempts=?, state=? WHERE challenge_id=?',
              (attempts, 'locked' if locked else 'open', challenge_id))
    if locked:
        raise OtpError(410, "Too many wrong codes. Please sign in again.")
    raise OtpError(401, f"Incorrect code. {MAX_ATTEMPTS - attempts} attempts left.")
```

5.3 A defect found by the test suite

The first version of the OTP module passed its logic review but failed test F4 ("three wrong codes lock the challenge"). The cause was subtle: the attempt counter was updated inside a database context manager, and the handler then raised an HTTP error. The standard SQLite context manager rolls back when an exception leaves it, so every failed guess was recorded and then undone, and the attempt limit never took effect. A voter, or an attacker holding a password, could have guessed codes without limit. The fix is a small context manager that commits state changes before the exception propagates. We include this case because it shows the value of executable tests over review for security logic, and because the defect would not have been visible in normal use.

5.4 Running the system

Setup is three steps: install the requirements and build the front end once, copy `.env.example` to `.env` and fill in the values needed, and start the server. Existing ledger entries can be copied to MongoDB with `migrate_ledger_to_mongo.py`, which refuses to overwrite a non-empty collection. The experiment suite (`experiments/run_experiments.py`) runs against a throwaway copy and never touches real data (Appendix C).

6. Evaluation

6.1 Method

We evaluate the system with an executable suite (`experiments/run_experiments.py`) rather than by argument. The suite copies the application into a temporary directory, so it never touches real data, uses the file ledger so it never touches Atlas, drives the real HTTP API through FastAPI's

test client, and models a rogue administrator by editing the SQLite file and the ledger file directly, with foreign keys off, as a database console would. It has five groups:

- **A. Database-level prevention** (7 tests): statements that should be rejected by the schema.
- **B. API abuse** (13 tests): wrong passwords, injection, replay, cross-election ranks, concurrency.
- **C. Tampering** (13 scenarios): edits by someone with direct storage access, each followed by the three checks an auditor has: the integrity sweep, the chain verification, and the three headline counters shown in the administrator interface.
- **D. Privacy structure**: an automated check that ballots cannot be linked to voters by schema.
- **E. OTP** (14 tests) and **F. Benchmarks** at 100, 1,000 and 10,000 ballots.

Each tampering scenario is run twice. The **baseline** is the plain hash-chain design (plain SHA-256 chain, two-rule sweep) that represents the conventional approach [10]-[12]; it is the earlier version of PolyVote, checked out from version control and run unchanged. The **final** design is the current code with the keyed chain enabled. The attacker scripts never receive the key. A scenario counts as *Detected* if the sweep or the chain check reports it, as *Counters only* if the only evidence is a disagreement between the three counters that an officer would have to notice by eye, and as *Not detected* otherwise. Benchmarks are the median of three repeated runs. Tampering outcomes were identical across the repeated runs.

6.2 Prevention at the database layer

ID	Attempt	Mechanism	Result
A1	Mark an already-voted voter as voted again	trigger	Prevented
A2	Insert a rank outside 1-3 (rank 4)	CHECK	Prevented
A3	Rank the same candidate twice on one ballot	composite PK	Prevented
A4	Rank a candidate that does not exist (FK enforcement on)	FK	Prevented
A5	Rank a nonexistent candidate from a default connection (FK pragma off)	FK	Not prevented
A6	Register the same reg. number twice in one election	UNIQUE	Prevented
A7	Set has_voted to an invalid value (2)	CHECK	Prevented

Table 8: Statements submitted directly to the election database.

Six of the seven statements are rejected by the engine with its own error message. The exception, A5, is informative: SQLite enforces foreign keys only on connections that enable the setting, and a console opened with default settings does not. A rogue administrator can therefore insert a ballot that references a candidate that does not exist. Application connections enable the setting, and the extended sweep reports such a ballot as injected because it has no receipt, but the finding is a reminder that a constraint is only as strong as the engine's enforcement. A server database where the setting cannot be disabled per connection, which is the intended move to Oracle or PostgreSQL, removes this gap [29].

6.3 Abuse through the API

ID	Attempt	Expected	Result
B1	Log in with a wrong password	rejected (401)	Prevented
B2	Log in with a reg. number not on the roster	rejected (404)	Prevented
B3	SQL injection in the reg. number field	rejected	Prevented
B4	Replay a session token after the ballot was cast	rejected (401)	Prevented
B5	Log in again after voting	rejected (409)	Prevented
B6	Cast a ballot with no session	rejected (401)	Prevented
B7	Use a voter token on an admin endpoint	rejected (403)	Prevented
B8	Rank another election's candidates	rejected (422)	Prevented
B9	Rank the same candidate twice via the API	rejected (422)	Prevented
B10	Submit an incomplete ranking	rejected (422)	Prevented
B11	20 simultaneous casts from the same voter	exactly one accepted	Prevented
B12	Log in after the election is closed	rejected (409)	Prevented
B13	Re-open a closed election	rejected (409)	Prevented

Table 9: Requests made against the public API.

All thirteen are refused. B11 is the one that depends on concurrency control: twenty browser sessions of the same voter, opened before any of them voted, submit at the same instant behind a barrier, and exactly one ballot is stored. The write lock and the `has_voted` check inside the transaction together prevent the race that would otherwise let several of them pass the check at once.

6.4 Tampering by someone with storage access

This is the central experiment. Table 10 lists the thirteen scenarios and the outcome under each design, and Figure 6 summarises them.

ID	Scenario	Baseline (plain chain)	PolyVote (final)
C1	Swap the 1st and 2nd choice on one ballot	Detected	Detected
C2	Replace the first-choice candidate on one ballot	Detected	Detected
C3	Delete one ballot	Detected	Detected
C4	Delete a ballot and clear a voted flag to hide the gap	Detected	Detected
C5	Inject a ballot with no receipt	Counters only	Detected
C6	Ballot stuffing: inject ballot, forge receipt, set a voted flag	Not detected	Detected
C7	Edit a ledger block without repairing the chain	Detected	Detected
C8	Delete a block from the middle of the ledger	Detected	Detected
C9	Truncate the last block of the ledger	Counters only	Detected
C10	Change a ballot and rebuild the whole chain consistently (attacker has no secret key)	Not detected	Detected

ID	Scenario	Baseline (plain chain)	PolyVote (final)
C11	Swap two candidates' names after voting	Not detected	Detected
C12	Reset a voted flag, then let that voter cast a second ballot	Counters only	Detected
C13	Add a ghost voter to the roster and vote as them through the API	Not detected	Not detected

Table 10: Outcome of each tampering scenario under the two designs.

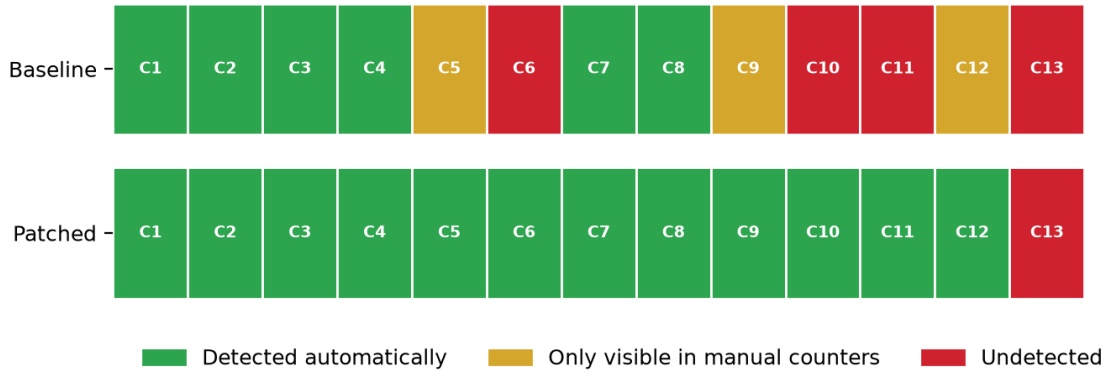


Figure 6: Detection outcome per scenario: plain hash-chain baseline versus PolyVote.

In plain words. Each square is one attack, C1 to C13. Green means PolyVote found it automatically, yellow means it was only visible if someone compared the numbers by eye, and red means it was not found. The top row is the plain hash-chain baseline (6 green) and the bottom row is PolyVote (12 green). The one red square left is C13, adding a fake voter to the roster, which we explain in Section 9.

Under the plain hash-chain baseline, 6 of 13 scenarios were detected automatically, 3 were visible only as counter mismatches, and 4 were not detected at all. Under the final design, 12 are detected automatically, none rely on the counters, and 1 is not detected. The scenarios behave as follows.

- **C1 to C4** (change, replace or delete ballots in the database): detected by both designs, because the sweep compares each ballot with its receipt. In C4 the attacker also clears a voted flag to hide the gap; the deleted ballot is still found.
- **C5** (inject a ballot with no receipt) and **C9** (truncate the last ledger block): the baseline sweep looked only at ballots that had a receipt, so neither was flagged; both were visible only because the ballot and receipt counts differ. The new "injected" rule and the reconciliation rule detect both automatically.
- **C6** (ballot stuffing: inject a ballot, forge its receipt, and set a spare voter's flag so all counters agree) and **C10** (alter a ballot and rebuild the whole chain consistently) were completely undetected by the baseline. They are exactly the attacks a plain hash chain cannot stop, because the attacker can recompute every hash. With the keyed chain the forged or rebuilt blocks fail verification at the first block whose hash the attacker could not compute.
- **C7, C8** (edit or delete a block): detected by both, by the chain check.

- **C11** (swap two candidates' names after voting): undetected by the baseline because names were outside the sealed data; the sealed roster hash now reveals it.
- **C12** (reset a voted flag so the voter can vote again): both ballots have valid receipts, so only the disagreement between flags, ballots and receipts exposes it. It was visible only to a careful reader in the baseline and is now reported by the sweep.
- **C13** (add a ghost voter to the roster and vote as them through the API) remains undetected. The ballot is valid, sealed and counted; what is false is the voter's eligibility, and the system holds no independent record of the roster to compare with. Closing this requires a signed or externally recorded roster, which we list as future work. We report it rather than hide it.

6.5 One-time-password verification

ID	Test	Expected	Result
F1	Correct password alone does not create a voting session	no token issued; challenge id is not a session	Passed
F2	Correct emailed code unlocks the ballot	session issued, ballot accepted	Passed
F3	A used code cannot be replayed	rejected (410)	Passed
F4	Three wrong codes lock the challenge	401, 401, 410 then the right code is refused	Passed
F5	An expired code is refused	rejected (410)	Passed
F6	Resend is rate-limited to one per 30 seconds	immediate resend 429, later resend 200	Passed
F7	A resent code invalidates the previous one	old code rejected	Passed
F8	A challenge allows at most 5 codes	sixth send refused (429)	Passed
F9	A voter with no email on file cannot proceed	409 with a clear message	Passed
F10	Plaintext codes are absent from the verification database	code not found in any table	Passed
F11	Repeated sign-ins are rate-limited	sixth challenge in 15 min refused (429)	Passed
F12	Codes are 6 digits including leading zeros	all length 6, mostly distinct	Passed
F13	A failed email send does not break sign-in	challenge still issued; failure logged; code stays in the outbox	Passed
F14	Voter email is not stored in the election database	no email column in any election table	Passed

Table 11: Tests of the email one-time-code module.

All fourteen pass. F1 and F2 establish the basic contract: a correct password alone yields no session and the challenge identifier is not accepted as a bearer token, while the correct code does open the ballot. F3 to F8 cover replay, guess limits, expiry, resend limits and the cap on codes per challenge. F10 and F14 are privacy properties: no plaintext code is stored in the verification database, and the election database has no email column. F13 injects an unreachable mail server and confirms sign-in is not blocked and the failure is logged. F4 initially failed and exposed the rollback defect described in Section 5.3.

6.6 Voter and ballot separation

Check	Result
Columns shared by `BALLOT_RANKS` and `VOTER_PARTICIPATION`, excluding `election_id`	none
Fields stored in a ledger block	`ballot_token`, `curr_hash`, `election_id`, `index`, `prev_hash`, `roster_hash`, `timestamp`, `vote_data_hash`
Voter email stored in the election database	no (F14)
Voter identifier present in any ledger block	no

Table 12: Automated check of the voter/ballot separation.

A new field, `roster_hash`, was added to ledger blocks in the final design. It is a hash of the candidate list and carries no information about any voter.

6.7 Performance

All benchmarks use synthetic elections with five candidates. Ballots are seeded directly into the database and the ledger so that large sizes can be built quickly; the timed operations are the real ones. Results are medians of three runs on a laptop.

Ballots	Cast (ms)	Sweep (ms)	Chain verify (ms)	Tally (ms)	Receipt lookup (ms)	File (KB)
100	9.38 / 10.0	1.82 / 4.19	0.82 / 1.75	8.16 / 11.69	4.45 / 10.06	48.0
1,000	33.48 / 28.52	16.98 / 11.86	7.36 / 9.7	10.95 / 8.75	7.77 / 7.2	448.8
10,000	204.85 / 207.49	128.74 / 147.41	80.4 / 102.5	57.2 / 60.21	42.53 / 49.21	4,465.4

Table 13: File-ledger timings, baseline / PolyVote (median of three runs).

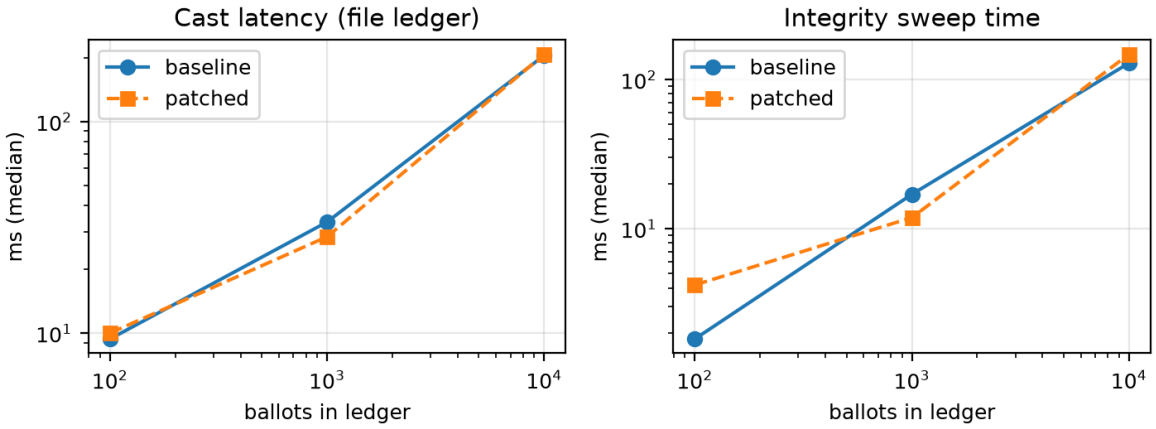


Figure 7: Cast latency and sweep time against ledger size (log scales).

In plain words. The left graph shows how long it takes to cast one ballot as the ledger grows. The right graph shows how long the integrity check takes. Both scales are logarithmic, so each step to the right is ten times more ballots. The two lines (old and final design) sit almost on top of each other, which shows the extra protection costs very little time.

Three observations follow. First, every read-side operation grows linearly with the number of ballots, which is expected from a design that recomputes rather than caches: at 10,000 ballots the sweep takes about 147 ms and the tally about 60 ms, which is acceptable for the target scale. Second, the extra checks and the HMAC add little: the sweep is roughly 15% slower and chain verification 27% slower at 10,000 ballots. Third, cast latency with the file ledger grows from 9.38 ms at 100 ballots to 205 ms at 10,000, because each append rewrites the whole JSON file. That growth is the practical argument for a real document store.

Ballots already in ledger	Append, median (ms)	Append, 90th percentile (ms)	Verify whole chain (ms)
100	135.9	157.4	231
1,000	147.2	170.4	703
10,000	139.4	163.5	3,046

Table 14: Ledger operations against MongoDB Atlas over a mobile hotspot (20 appends per row).

Against Atlas, appending costs about 147 ms whatever the chain length, because the append reads only the last block and inserts one document, so the cost is one or two network round trips. At small sizes the local file is faster, and the crossover is near ten thousand ballots on this network. The reasons to prefer Atlas are therefore not raw speed but flat growth, concurrency control through the unique index, and holding the ledger outside the machine that runs the application. Whole-chain verification reads every block over the network and grows with the chain, from about 0.2 s at 100 blocks to 3.0 s at 10,000. A production system would verify incrementally from a trusted checkpoint.

6.8 Application outputs

The figures below are screenshots from the running application. Names and addresses are demonstration values.

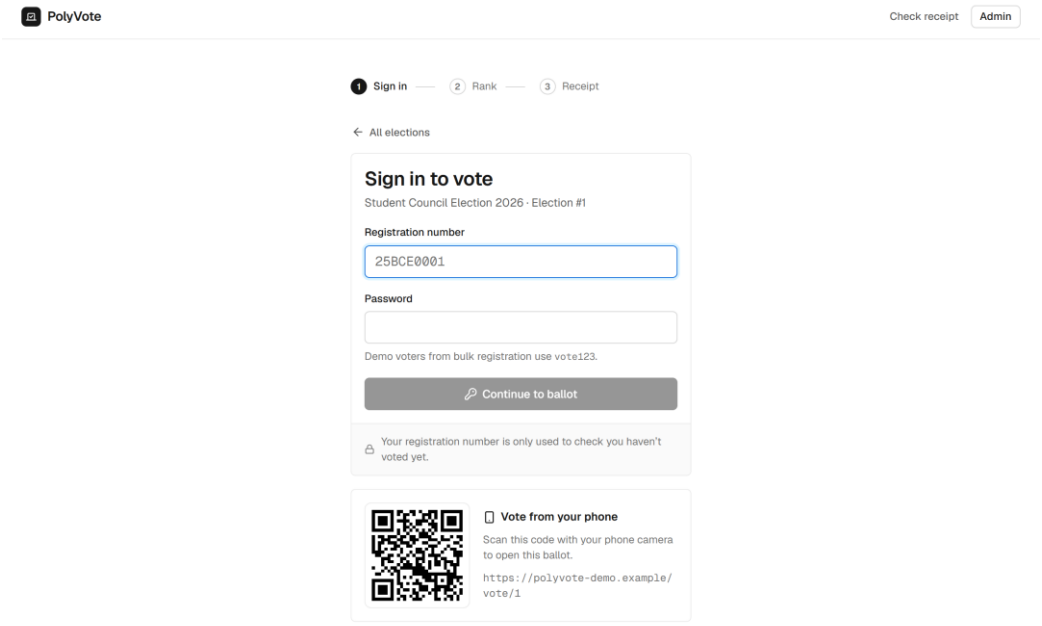


Figure 8: Voter sign-in, with the scan-to-vote QR code for phone voting.

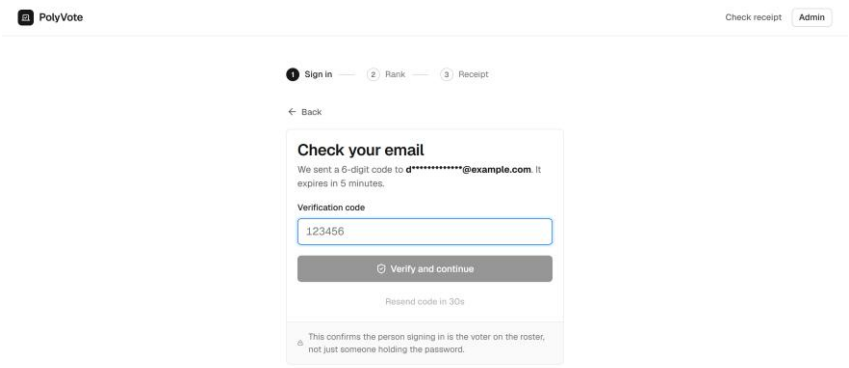


Figure 9: Second factor: the voter enters the six-digit code sent to the address on file.

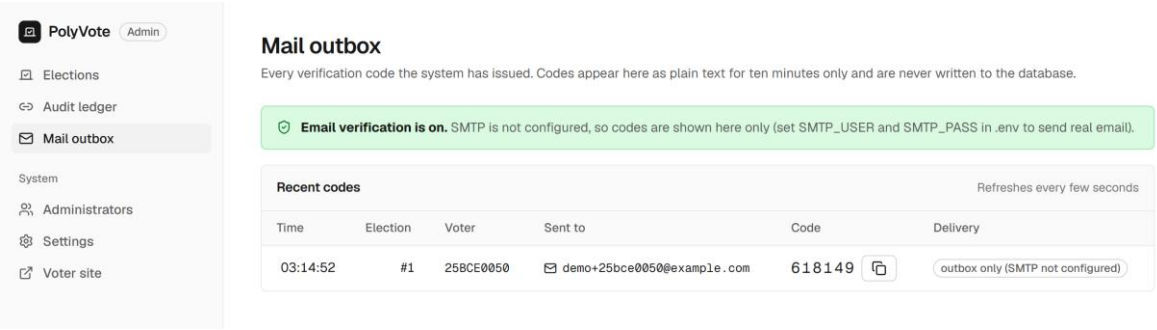


Figure 10: Administrator Mail outbox: each code issued, its recipient and delivery status.

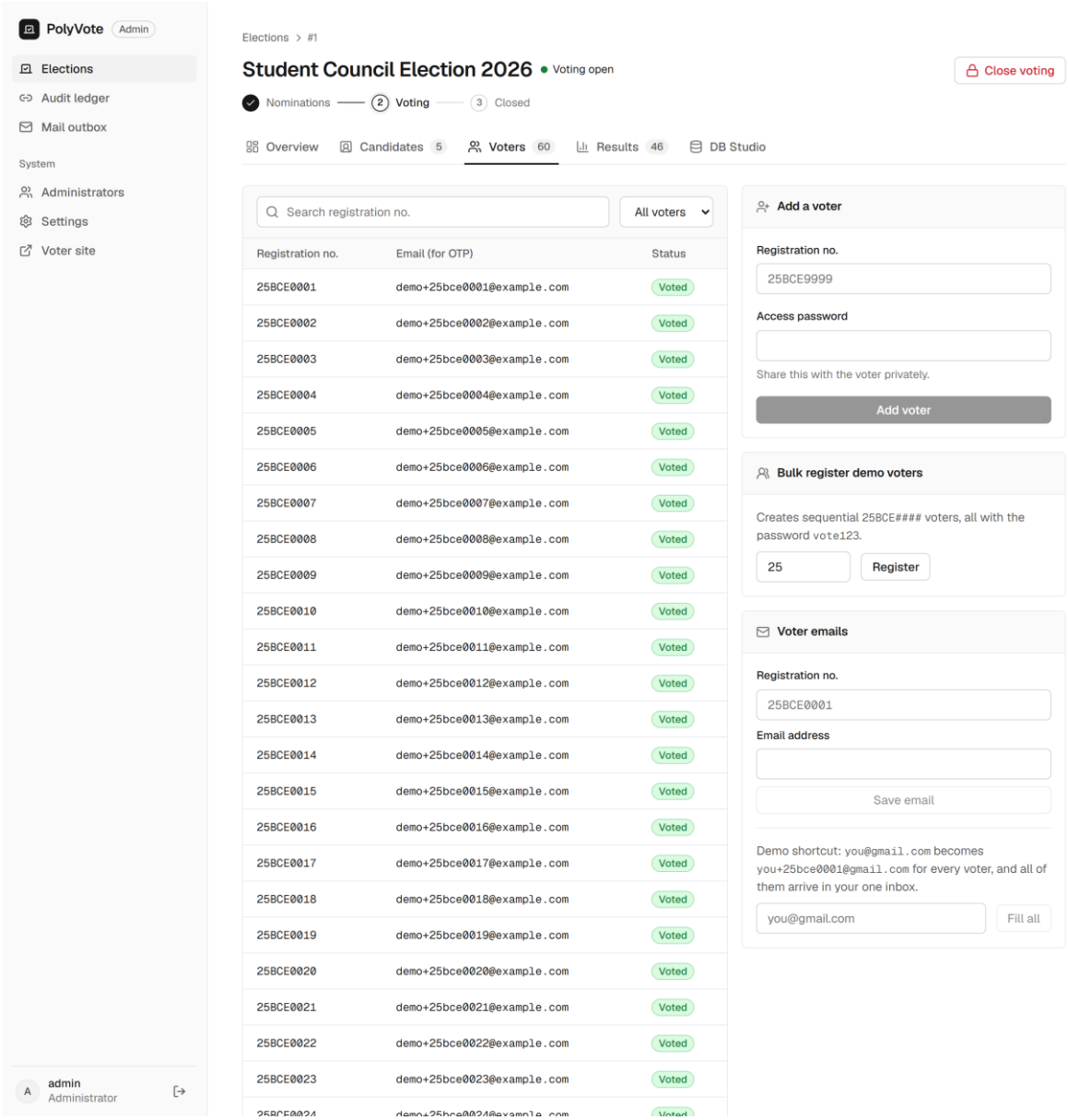


Figure 11: Voters tab with the email column and the bulk plus-address tool.

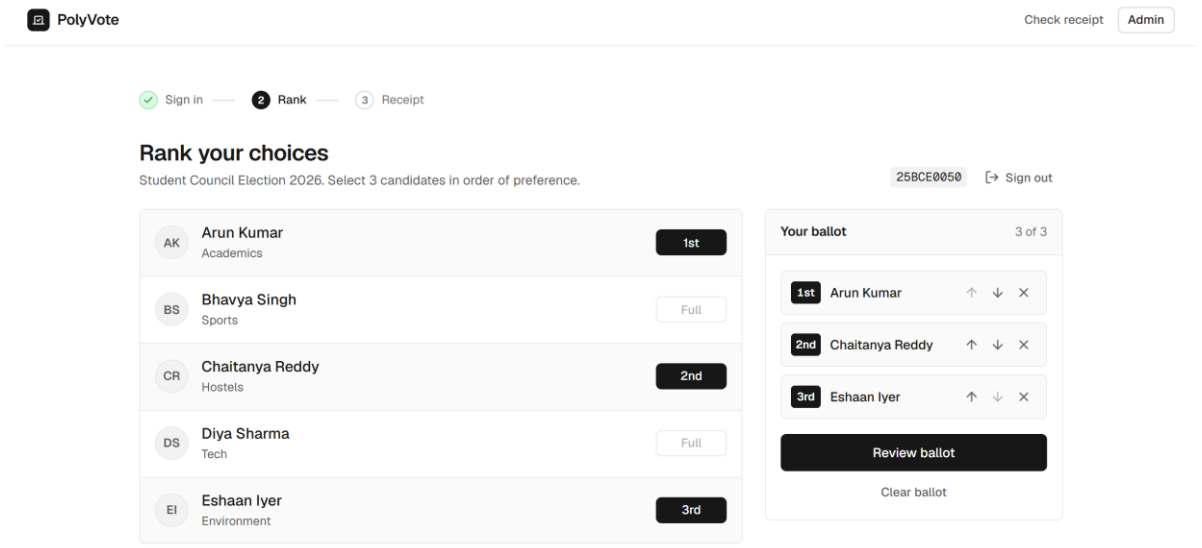


Figure 12: Ranking the ballot on a desktop.

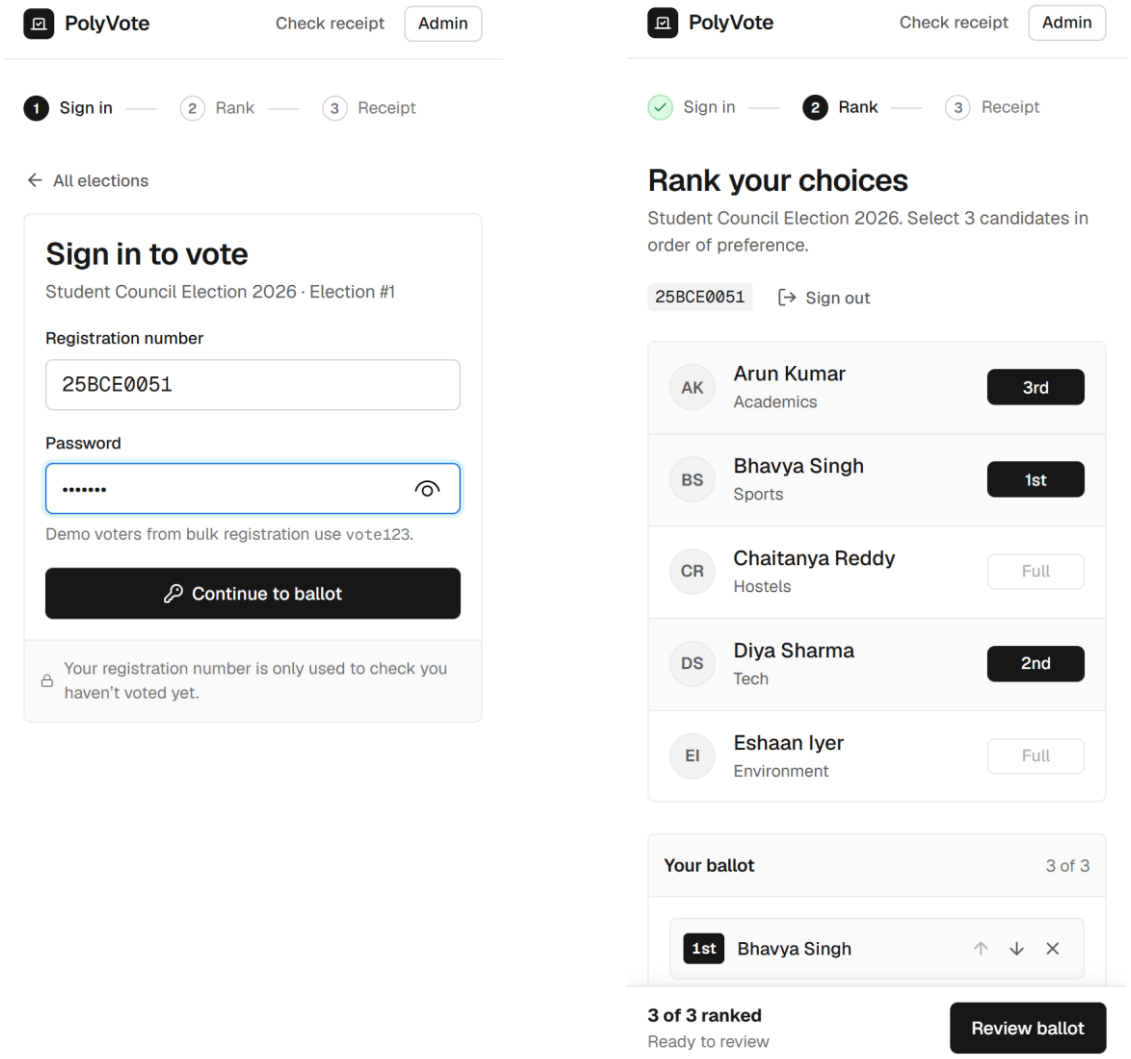


Figure 13: The same flow on a phone-sized screen.

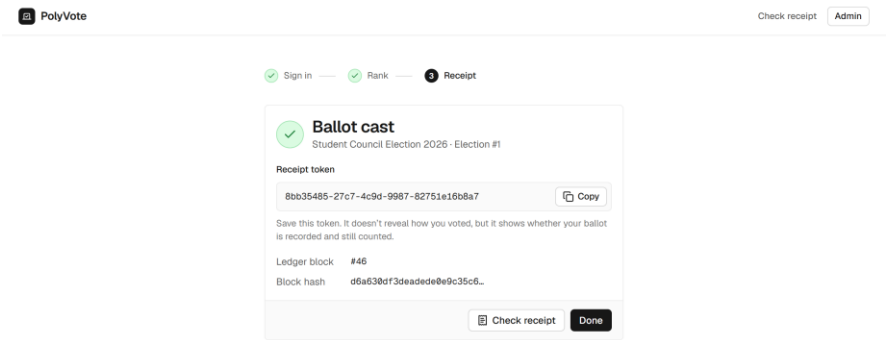


Figure 14: Receipt token and ledger block returned after the ballot is sealed.

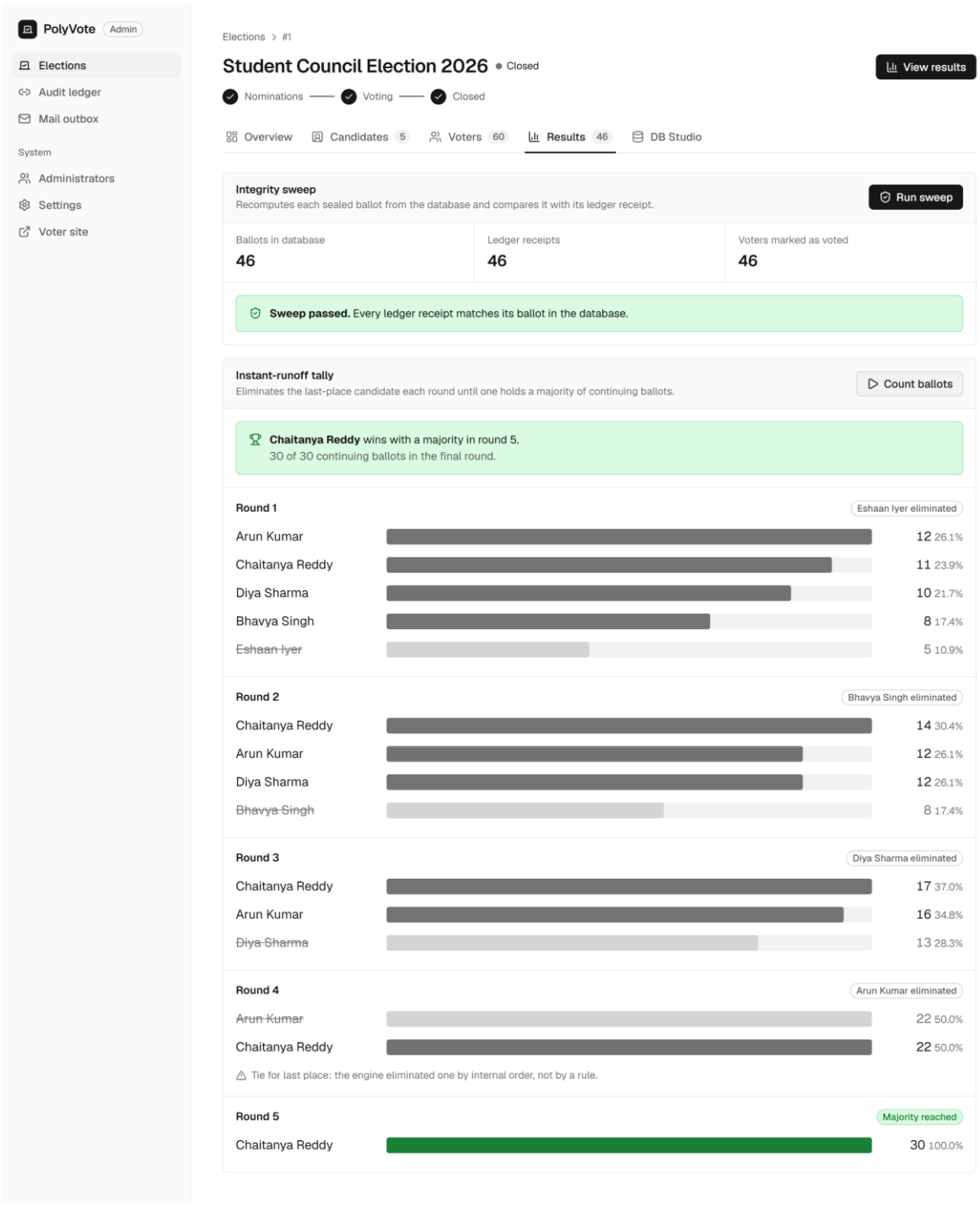


Figure 15: Results before tampering: sweep passed, instant-runoff rounds.

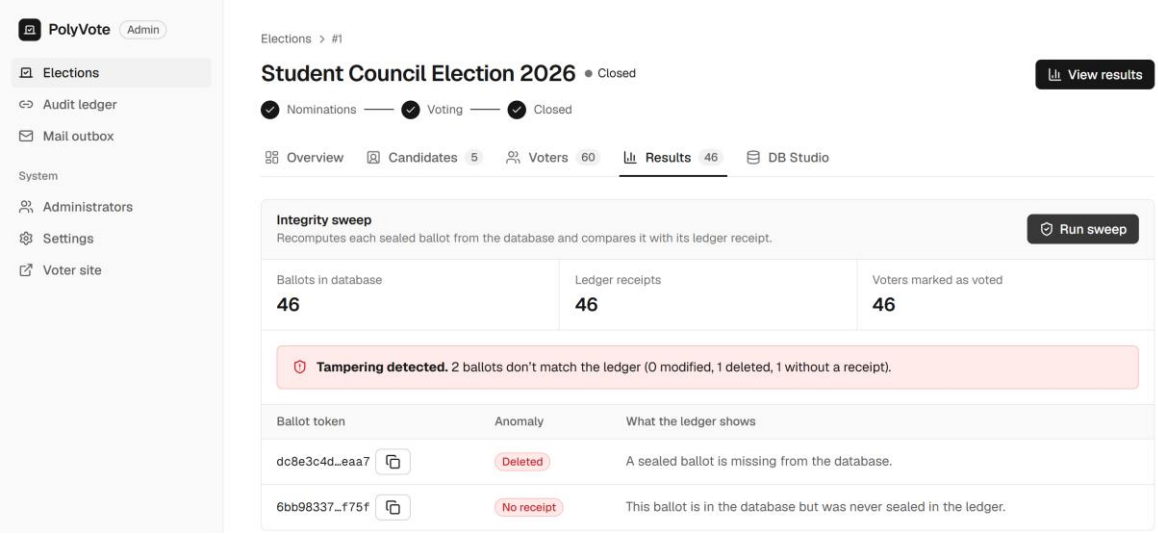


Figure 16: Sweep after one ballot was deleted and another injected: both are reported even though the three counters still read 46 / 46 / 46.

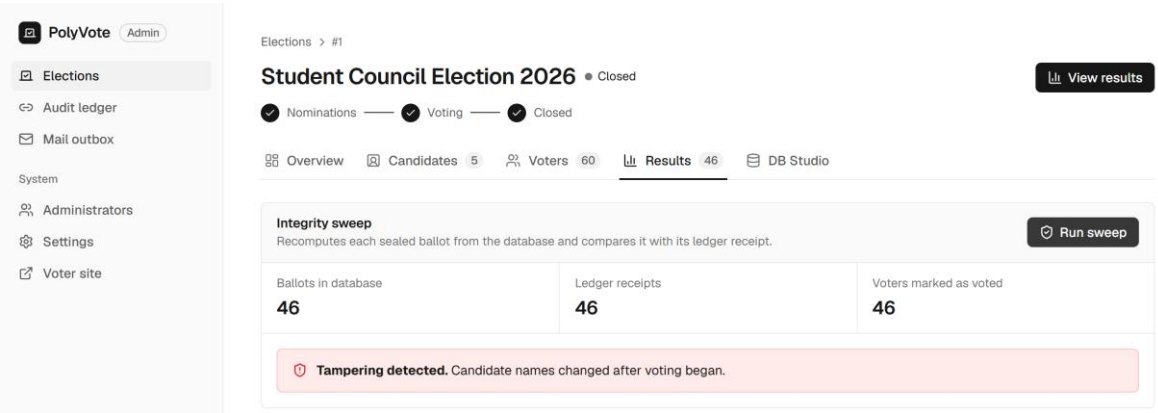


Figure 17: Sweep after two candidate names were swapped.

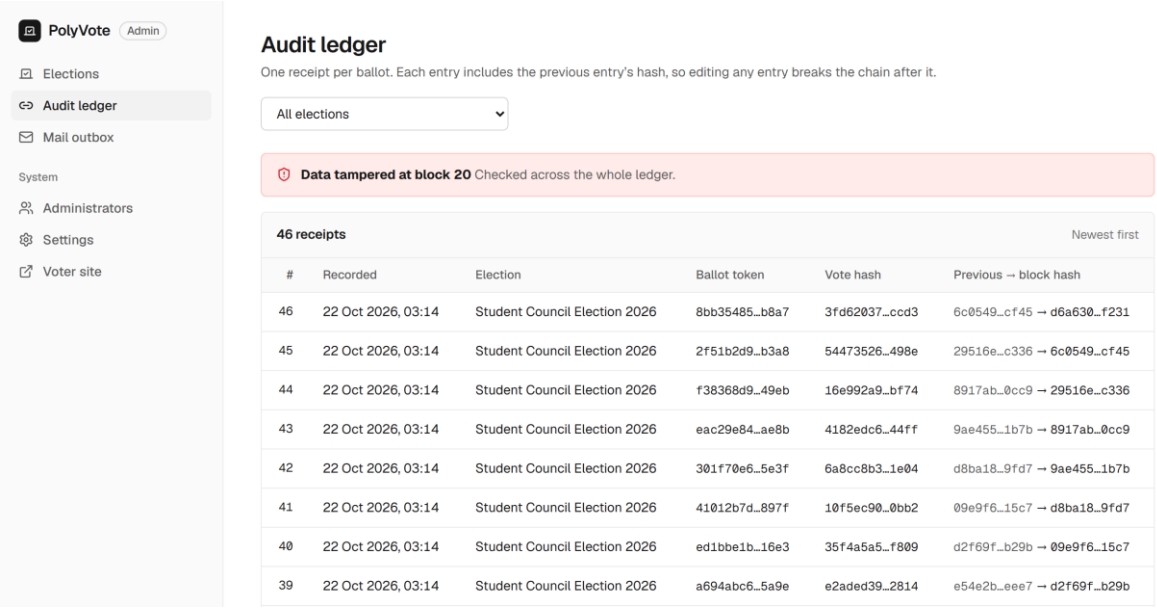


Figure 18: Audit ledger after a block was edited: the chain check fails.

6.9 Live demonstration

The final build, Sample PolyVote, has the email one-time code switched on and was demonstrated live in class. The demonstration went through the complete voter journey and it worked end to end. Table 15 lists each step, what was shown and the screenshot that serves as evidence.

Step	What was shown	Result	Evidence
1	Officer adds voters and their email addresses	Roster with an Email column; emails stored in a separate database	Figure 11
2	Voter opens the vote page; QR code shown	Phone opens the ballot by scanning the QR code	Figure 8, Figure 13
3	Voter signs in with the correct password	No ballot yet: the page asks for the emailed code	Figure 9
4	Code is issued and sent	Code appears in the Mail outbox and arrives by email	Figure 10
5	Voter enters the code and ranks three candidates	Ballot screen opens only after the right code	Figure 12
6	Voter casts the ballot	Receipt token and ledger block returned	Figure 14
7	Officer closes the election, runs the sweep and counts	Sweep passes; instant-runoff rounds shown	Figure 15
8	Tamper test: ballot deleted and another injected	Sweep names both even though the counters still match	Figure 16
9	A ledger block is edited	Chain check shows a red "tampered" banner	Figure 18

Table 15: Steps of the live demonstration and the evidence for each.

The screenshots in Figures 8 to 18 come from the same build that was demonstrated. They show the one-time code stopping a password-only sign-in, the QR code opening the ballot on a phone, the receipt after voting, and the sweep and chain check catching tampering.

6.10 Threats to validity

- **Synthetic workload.** Elections are generated; real turnout patterns and larger rosters might differ. We used up to 10,000 ballots, which exceeds any single election we target.
- **One machine and one network.** Timings come from a laptop, and Atlas timings from a mobile hotspot, so absolute values will differ elsewhere. We report ratios and trends.
- **Attacker models are ours.** Scenarios C1 to C13 are the attacks we thought of, expressed against our own design. They are not a complete catalogue, and an author tends to test what the design handles.
- **The baseline is our own earlier version.** We did not run Helios, Belenios or the commercial systems, so the comparison with them in Section 7 is qualitative and based on their published designs.
- **Detection depends on running the sweep.** The system detects tampering when the officer runs the checks; it does not prevent it or alert automatically.

7. Comparative Study

PolyVote is a combination of parts, so we compare it part by part with the related projects and technologies that do the same job. For every part we show what the others offer, what PolyVote achieved in our tests, and where the proof is in this report. We separate two kinds of evidence. **Measured evidence** comes from our own tests and benchmarks (Section 6) and is exact. **Published evidence** comes from the papers and vendor pages in the reference list; we did not run those systems, so statements about them are only as strong as their sources.

7.1 Method and verdict rule

- **Seven parts:** integrity rules inside the database, voter/ballot separation, the audit ledger, voter authentication, vote counting, storage architecture, and access (phone, QR code, setup).
- **Verdict rule:** we write *Better* only where a test or benchmark in this report shows it. We write *Comparable* where the mechanism matches the alternative, and *Not better* where the alternative is stronger. Nothing is claimed without a table or figure to point to.
- **Baseline:** for the ledger part we also measure a plain hash-chain design (the conventional approach in [10]-[12]) with the same attacks, so the comparison there is measured, not argued.

7.2 Part 1: integrity rules inside the database

The job: refuse a second vote, a bad rank or a ballot for a missing candidate.

Compared with	What it offers (source)	PolyVote: proven result	Evidence
Form tools and typical student apps	One vote per person depends on sign-in or application code; the owner or administrator can edit stored answers	6 of 7 database statements rejected by the engine; 13 of 13 API abuse tests refused; 20 simultaneous casts stored exactly 1 ballot	Table 8, Table 9

Compared with	What it offers (source)	PolyVote: proven result	Evidence
Server database constraints and triggers [29], [30]	The same mechanisms, enforced on every connection	The same mechanisms (trigger, CHECK, UNIQUE, foreign key). Gap found: SQLite enforces foreign keys only when switched on per connection (test A5)	Table 8, test A5

Table 16: Part 1 compared with related approaches.

Verdict. *Better* than apps that enforce the rule only in application code (measured). *Comparable* to server databases in mechanism; *not better* on foreign-key enforcement until a server database is used.

7.3 Part 2: voter and ballot separation

The job: make sure nobody can look up who voted for whom.

Compared with	What it offers (source)	PolyVote: proven result	Evidence
Helios [1], Belenios [2]	Ballots are encrypted; privacy depends on the server and trustees	No column links a ballot to a voter (automated check found none); voter emails are in a separate database (test F14)	Table 12, test F14
Commercial hosted voting	Storage design is not published	Cannot be compared; we show our own evidence only	Table 12

Table 17: Part 2 compared with related systems.

Verdict. *Comparable* for a small election: privacy comes from how the data is stored, which we tested. *Not better* than encryption: a voter who shows their receipt token reveals their ballot to anyone with database access (Section 9).

7.4 Part 3: the audit ledger

The job: prove afterwards that stored ballots were not edited, deleted or added.

Compared with	What it offers (source)	PolyVote: proven result	Evidence
Plain hash chain [10], [11] (our baseline)	Detects edits inside the log, but an attacker who can write the log can recompute it	Baseline found 6 of 13 attacks automatically; a forged receipt (C6) and a consistent rewrite (C10) went unnoticed	Table 10, Figure 6
Merkle-tree logs [12], [13]	Short proofs that an entry is in the log; heavier to build	PolyVote (keyed chain plus sweep) found 12 of 13; C6 and C10 now caught. Whole-chain check reads every block: 3.0 s at 10,000 blocks on Atlas	Table 10, Table 14
Ledger databases such as SQL Ledger [26]	Verifiable history built into the database engine	Similar goal using MongoDB Atlas and a sweep; cost of the extra protection: sweep +15% at 10,000 ballots	Table 13
Blockchain voting [14]	Criticised for adding risk without solving core voting problems	PolyVote uses no blockchain; one keyed chain with the key kept outside the stores	Section 2.2

Table 18: Part 3 compared with related approaches.

Verdict. *Better* than a plain hash chain (measured: 6 to 12 of 13). Simpler than Merkle logs but *not better*: checking is linear in the chain length, and anyone holding the key can still forge blocks.

7.5 Part 4: voter authentication

The job: make sure the person voting is the person whose password is used.

Compared with	What it offers (source)	PolyVote: proven result	Evidence
Password only (form tools, typical student apps)	A leaked or shared password is enough	A correct password alone gives no session; the emailed code is also needed (tests F1, F2)	Table 11
NIST guidance [31]	Recommends a second factor held by the user	14 of 14 tests: replay refused, lock after 3 wrong codes, expiry, resend and rate limits	Table 11
Email and phone channels [36], [37]	Convenient, but only as strong as the mailbox or device	Same limit applies; a mail failure never blocks sign-in (test F13); code also visible in the admin outbox	Table 11, Figure 10
Hardware tokens and authenticator apps [32], [33]	A stronger second factor	Not implemented (future work)	Section 9

Table 19: Part 4 compared with related approaches.

Verdict. *Better* than password-only sign-in (14 of 14 tests, and the tests also found and fixed a real defect, Section 5.3). *Not better* than hardware tokens.

7.6 Part 5: vote counting

The job: produce a winner with majority support and show how it was reached.

Compared with	What it offers (source)	PolyVote: proven result	Evidence
Plurality tools (forms, many student apps)	The winner can lack majority support	Instant-runoff rounds with a full transcript; ties for last place are flagged	Figure 15
Audit research for ranked ballots [15]-[18]	Margin computation and risk-limiting audits	Not implemented: PolyVote counts and reports, with no statistical audit	Section 9

Table 20: Part 5 compared with related approaches.

Verdict. *Better* than plurality counting on the same ballots (the rounds are shown). *Not better* than audit tools, which PolyVote does not offer.

7.7 Part 6: storage architecture

The job: keep ballots, emails and the ledger in the right kind of store and keep it fast as it grows.

Compared with	What it offers (source)	PolyVote: proven result	Evidence
Single-file or single-store designs	Ledger and data live in one place	File ledger: cost of one cast grows from 9 ms to 205 ms between 100 and 10,000 ballots	Table 13
Polyglot persistence [21], [22]; shared multi-tenant schema [19], [20]	Different stores for different data; one schema for many tenants	MongoDB Atlas append about 147 ms at any size; several elections and organisations ran in one install in the tests	Table 14, tests B8, B13

Table 21: Part 6 compared with related approaches.

Verdict. *Better* than a single file ledger once ballots grow (crossover near 10,000); for a very small election the file is faster. Multiple tenants and stores were exercised in the tests.

7.8 Part 7: access and ease of running

The job: let voters join easily from any device and let a small team run the election.

Compared with	What it offers (source)	PolyVote: proven result	Evidence
Estonian i-voting [6]	National ID card and installed software	A web page opened by scanning a QR code; nothing to install; demonstrated from a phone	Table 15, Figure 8, Figure 13
Voatz [7], Democracy Live [8]	Mobile or web voting products; independent analyses found serious vulnerabilities	PolyVote has had no independent security analysis; we ran 47 tests ourselves	Table 8 to Table 11
Helios [1]	Needs key set-up and audit participation	One command starts the system; set-up steps in Section 5.4	Section 5.4

Table 22: Part 7 compared with related systems.

Verdict. *Better* for a small team to run and for voters to join (demonstrated), with the caveat that internet voting in general remains contested [9].

7.9 Side-by-side comparison with CRUD apps, form tools, SaaS and cryptographic systems

A typical student voting project is a CRUD application: one table of votes, one password per user and no audit trail. Form tools (such as Google Forms) and commercial hosted voting services (SaaS) are the other things people actually use for small elections, and Helios and Belenios are the cryptographic reference points. We compare PolyVote with all four, first with a measurement and then property by property.

Measurement. Eight of our thirteen tampering scenarios apply to every design because they only change stored votes or voter flags (C1 to C5, C11, C12, C13); the other five (C6 to C10) forge or edit ledger entries, which a CRUD app does not have. We scored each design on those eight using only the checks it has. A typical CRUD app can compare its voted flags with its ballot count. The plain hash-chain baseline adds a receipt per ballot and a two-rule sweep. PolyVote adds the keyed chain and five sweep rules.

Design	Checks it has	Detected of the 8 shared scenarios	Which ones
Typical CRUD app	Voted flags compared with the ballot count	3 of 8	C3, C5, C12
Plain hash-chain baseline	Receipt per ballot, two-rule sweep, chain check	4 of 8 automatically	C1, C2, C3, C4
PolyVote (final)	Keyed chain, five-rule sweep, roster seal	7 of 8 automatically	C1, C2, C3, C4, C5, C11, C12

Table 23: Measured detection on the scenarios that apply to every design.

A CRUD app notices only the attacks that change a count (C3, C5, C12). It misses edited ballots (C1, C2), a deletion that is hidden by clearing a flag (C4), and renamed candidates (C11). The baseline catches edited and deleted ballots but not renamed candidates or ghost voters; PolyVote adds the roster seal and the extra rules. The one scenario nobody here catches is C13, a ghost voter added to the roster.

Property	CRUD app	Form tools	Commercial SaaS	Helios / Belenios	PolyVote	PolyVote evidence
Ranked-choice counting	No	No	Advertised	Varies	Yes	Figure 15
Many organisations	No	No	Yes	Partly	Yes	Tests B8, B13

Property	CRUD app	Form tools	Commercial SaaS	Helios / Belenios	PolyVote	PolyVote evidence
One-vote rule in the database	Rare	No	Not documented	n/a (crypto)	Yes	Table 8, test B11
Voter/ballot separation	Rare	No	Not documented	Yes [1], [2]	Yes	Table 12
Tamper detection	Counts only (3 of 8)	No	Vendor logs	Yes [1], [2]	7 of 8	Table 23
Second factor at sign-in	Rare	Account login	Often email links	Credential email	Emailed OTP	Table 11
Scan-to-vote on a phone	No	No	Varies	No	Yes	Table 15
Voter verifies ballot content	No	No	Varies	Yes [1], [2]	No	Section 9
Open source, one command	Often	No	No	Yes	Yes	Section 5.4
Cost for a small election	Free	Free	Paid per voter	Free, operations-heavy	Free	Section 4.10

Table 24: Property comparison; entries for other systems come from published designs and vendor pages (marked "Varies" or "Not documented" where we could not confirm).

7.10 Part-by-part summary

Part	Compared with	PolyVote proven result	Evidence	Verdict
1 Integrity rules	Form tools; server databases	6/7 DB tests, 13/13 API tests, 1 of 20 simultaneous casts	Table 8, Table 9	Better than apps; comparable to server databases
2 Voter/ballot separation	Helios, Belenios	No shared column; email in separate store	Table 12	Comparable; weaker than encryption
3 Audit ledger	Plain hash chain; Merkle logs; ledger databases	12 of 13 attacks found vs 6 for plain chain	Table 10	Better than plain chain; simpler than Merkle logs
4 Authentication	Password only; hardware tokens	14 of 14 OTP tests	Table 11	Better than password only; weaker than tokens
5 Counting	Plurality; audit tools	Instant-runoff transcript, ties flagged	Figure 15	Better than plurality; no audit
6 Storage	Single file ledger	Atlas about 147 ms flat vs file 205 ms at 10,000	Table 14	Better at scale
7 Access	National and mobile apps; Helios	QR on a phone; one-command start; live demo	Table 15	Easier to run and join

Table 25: Part-by-part comparison and verdicts.

7.11 What is unusual about the combination

We looked for a small-election system that combines all seven parts and publishes test evidence for each. We found strong systems for individual parts (cryptographic verifiability, hosted convenience, tamper-evident logs), but none that combines ranked-choice counting, rules enforced inside the database, a keyed audit ledger in a separate store, an emailed second factor and phone voting in one install that anyone can run with one command, with test results for every claim. In that sense PolyVote is unusual. We cannot prove that no such system exists, so we state it as what we found, not as a guarantee.

7.12 Conclusions drawn from the evidence

#	Conclusion	Evidence	Limit
C-1	PolyVote finds twice as many tampering attacks automatically as a plain hash chain.	6 to 12 of 13 attacks, identical across 3 runs (Table 10)	Attacks are the ones we designed; one (C13) is still missed.
C-1b	A typical CRUD voting app notices only the tampering that changes a count.	3 of 8 shared scenarios (Table 23)	The CRUD model is our reasoned minimal design, not a specific product.
C-2	A keyed chain stops attacks that a plain hash chain cannot.	C6 and C10 undetected by the baseline, detected by PolyVote	Whoever holds the key can still forge.
C-3	The sweep catches changes that the counters hide.	Figure 16: deletion plus injection kept counters at 46/46/46 yet both were named	Needs the officer to run the sweep.
C-4	Email OTP makes a stolen password insufficient and resists guessing.	14 of 14 tests (F1 to F14), including limits and replay	Email is a weaker factor than a hardware token.
C-5	The extra protection costs little time.	Sweep +15% at 10,000 ballots; Atlas append flat	Synthetic data on one laptop.
C-6	The whole system works end to end, including on a phone.	Table 15 and the live demonstration	Small demonstration group.
C-7	PolyVote is not an end-to-end verifiable system.	Voter cannot check ballot content (Section 9)	Helios and Belenios are stronger on this one property.

Table 26: Conclusions, their evidence and their limits.

Overall finding. Part by part, PolyVote is better than the simpler alternatives on integrity rules, the audit ledger, authentication, counting, storage at scale and ease of access, comparable on voter privacy, and weaker than cryptographic systems on ballot verifiability. For small and medium elections that need ranked-choice results, tested tamper detection and a one-command setup, it is the better fit; it is not a substitute for cryptographic systems where the administrator cannot be trusted or voters must verify their own ballot content.

8. Final Implementation Status

8.1 What is implemented

Everything in the project plan is implemented and was exercised in the tests and the live demonstration. Table 27 lists each component and the evidence.

Component	Status	Evidence
Normalised multi-tenant relational schema, constraints, trigger and views	Implemented	Table 8, Table 3
Voter and admin web application (React and FastAPI), including QR-code voting	Implemented	Table 15, Figure 8
Instant-runoff counting with a round-by-round transcript	Implemented	Figure 15
Keyed audit ledger in MongoDB Atlas with file fallback	Implemented	Table 14, Figure 18
Integrity sweep with five rules	Implemented	Table 10, Figure 16
Email one-time-code verification, voter emails and Mail outbox	Implemented	Table 11, Figure 9, Figure 10
Public HTTPS link: tunnel for live demos; Docker image and free cloud demo at https://polyvote-bacse202.onrender.com with a /demo guide to every page	Implemented	Table 15; Section 4.10

Component	Status	Evidence
Executable test suite (47 scenarios) and benchmarks	Implemented	Table 13, Table 26
Live demonstration with email OTP and QR voting	Done, worked end to end	Table 15

Table 27: Implemented components and their evidence.

8.2 What was left out

The one deliberate substitution is the database engine. The plan named Oracle SQL with PL/SQL; we opted for SQLite for the relational layer. SQLite gives the same constraints, trigger and views that the project relies on, and the design is portable, but it has no PL/SQL stored procedures and no DCL roles, so those Oracle-specific features were not built. Two smaller extensions, several positions per election and a production-grade cloud deployment with persistent storage (the free demo host wipes its data on restart), are listed as future work in Section 9.

8.3 Mapping to course outcomes

The mapping below follows our reading of the course outcomes for BACSE202 and should be adjusted to the official wording.

Outcome	Evidence in this report
Design a normalised relational schema with constraints	Section 4.2; ER diagram; rules table; prevention tests A1 to A7
Use triggers, views and transactions for integrity	Sections 4.4 and 5.2; duplicate-vote trigger; atomic cast; concurrency test B11
Apply database security and access separation	Sections 3, 4.3 and 4.8; voter/ballot separation; verification database; OTP tests
Use a NoSQL store appropriately (polyglot persistence)	Sections 4.5 and 6.7; MongoDB Atlas ledger; benchmark against the file ledger
Evaluate a design empirically and compare with alternatives	Sections 6 and 7; 47 scenarios; part-by-part comparison with related projects
Communicate the work	This report, the review slides, the repository and the running public demonstration

Table 28: Course outcomes and where they are addressed.

9. Limitations and Future Work

Limitation	Why it matters	Direction
Host compromise or key theft (A6)	Whoever holds `LEDGER_KEY` and the stores can forge a consistent chain	Periodically publish the chain head to an external, append-only witness, in the manner of transparency logs [13]; keep the key in a managed secret store
Roster injection (A7, scenario C13)	A ghost voter's ballot is valid and sealed	Commit the roster at registration close, with a signature or dual control; compare at sweep time
Not end-to-end verifiable	A receipt shows a ballot is recorded and counted, not what it contains [38]	Add a cast-as-intended mechanism, or adopt a verifiable protocol where the threat model demands it [1], [2]

Limitation	Why it matters	Direction
Foreign keys depend on a per-connection setting (A5)	A console with default settings can insert dangling ballots	Move to Oracle or PostgreSQL, where constraints cannot be bypassed per connection [29]
Passwords stored as plain SHA-256; default administrator password	Fast, unsalted hashes are weak against offline guessing [34], [35]; demo voters share a password	Use a salted, slow key-derivation function; force a password change on first login
Email is a weak second factor	Whoever reads the mailbox passes the check [36], [37]	Offer an authenticator app (TOTP [33]) or a printed one-time slip as alternatives
Receipt token links to the ballot content in the database	A voter who shows the token under duress reveals the ballot to anyone with database access	Encrypt ranks to a key not held by the database administrator
IRV tie-breaking is arbitrary; no statistical audit	Close contests depend on an unprincipled rule [15], [16]	Define a published tie-break rule; add risk-limiting audit support
Ledger append follows the database commit	A crash between them leaves a ballot with no receipt	The sweep detects it; a write-ahead outbox would remove it
Single node, SQLite writer	Limits throughput and availability	A server database and a replicated service for larger elections
Whole-chain verification reads every block	Verification time grows with the chain	Verify incrementally from a trusted checkpoint; Merkle structures [12]
Oracle SQL not used	SQLite replaces Oracle SQL, so there are no PL/SQL procedures or DCL roles; several positions per election and a production-grade cloud deployment with persistent storage are also future work	Move the relational layer to Oracle with PL/SQL and roles, then deploy with persistent storage

Table 29: Known limitations and planned work.

10. Conclusion

PolyVote started as an election application whose integrity rested on trusting whoever could write to its database. This report shows how far that trust can be reduced using database features alone. Constraints and a trigger enforce one-person-one-vote and well-formed ballots in the engine; ballots and voters are stored without a linking column; a keyed hash-chained ledger in a separate document store seals each ballot; and a sweep that reconciles ballots, receipts, flags and the candidate roster exposes tampering. A second factor, a standalone emailed one-time code, addresses the weakness of password-only sign-in.

The evidence is executable. Of 13 tampering scenarios, a plain hash-chain baseline detected 6 automatically and the final design detects 12. The two attacks a plain hash chain cannot stop, forging a receipt and rebuilding the chain, are caught once the chain is keyed. The cost is small: the sweep is about 14% slower at 10,000 ballots. One scenario, roster injection, is not detected, and we have said so, along with the other limits of a design that is not end-to-end verifiable.

We therefore claim a specific improvement: over a plain hash-chain design and over unprotected relational election applications, PolyVote substantially raises the cost of undetected tampering at the database layer and strengthens voter authentication, while remaining simple to run. We do not claim parity with cryptographic voting systems. The only planned feature left out is Oracle SQL with PL/SQL, because we opted for SQLite. Future work is the move to Oracle, an external witness

for the chain head and a committed roster, is well defined and follows directly from the limitations reported here.

References

[1] B. Adida, “Helios: Web-based Open-Audit Voting,” in Proc. 17th USENIX Security Symposium (USENIX Security 08), San Jose, CA, 2008. [Online]. Available: https://www.usenix.org/conference/17th-usenix-security-symposium/helios-web-based-open-audit-voting

[2] V. Cortier, P. Gaudry, and S. Glondu, “Belenios: A Simple Private and Verifiable Electronic Voting System,” in Foundations of Security, Protocols, and Equational Reasoning (Lecture Notes in Computer Science), Springer, 2019, pp. 214–238. doi: 10.1007/978-3-030-19052-1_14.

[3] M. R. Clarkson, S. Chong, and A. C. Myers, “Civitas: Toward a Secure Voting System,” in Proc. 2008 IEEE Symposium on Security and Privacy, 2008, pp. 354–368. doi: 10.1109/sp.2008.32.

[4] P. Y. A. Ryan, D. Bismark, J. Heather, S. Schneider, and Z. Xia, “Prêt à Voter: a Voter-Verifiable Voting System,” IEEE Transactions on Information Forensics and Security, vol. 4, no. 4, pp. 662–673, 2009. doi: 10.1109/TIFS.2009.2033233.

[5] D. Chaum et al., “Scantegrity II: End-to-End Verifiability by Voters of Optical Scan Elections Through Confirmation Codes,” IEEE Transactions on Information Forensics and Security, vol. 4, no. 4, pp. 611–627, 2009. doi: 10.1109/tifs.2009.2034919.

[6] D. Springall et al., “Security Analysis of the Estonian Internet Voting System,” in Proc. 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS '14), 2014, pp. 703–715. doi: 10.1145/2660267.2660315.

[7] M. A. Specter, J. Koppel, and D. Weitzner, “The Ballot is Busted Before the Blockchain: A Security Analysis of Voatz, the First Internet Voting Application Used in U.S. Federal Elections,” in Proc. 29th USENIX Security Symposium (USENIX Security 20), 2020, pp. 1535–1553. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/specter

[8] M. A. Specter and J. A. Halderman, “Security Analysis of the Democracy Live Online Voting System,” in Proc. 30th USENIX Security Symposium (USENIX Security 21), 2021, pp. 3077–3092. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/specter-security

[9] National Academies of Sciences, Engineering, and Medicine, Securing the Vote: Protecting American Democracy. National Academies Press, Washington, DC, 2018. doi: 10.17226/25120.

[10] S. Haber and W. S. Stornetta, “How to time-stamp a digital document,” Journal of Cryptology, vol. 3, no. 2, pp. 99–111, 1991. doi: 10.1007/BF00196791.

[11] B. Schneier and J. Kelsey, “Secure audit logs to support computer forensics,” ACM Transactions on Information and System Security, vol. 2, no. 2, pp. 159–176, 1999. doi: 10.1145/317087.317089.

[12] S. A. Crosby and D. S. Wallach, “Efficient Data Structures for Tamper-Evident Logging,” in Proc. 18th USENIX Security Symposium (USENIX Security 09), Montreal, Canada, 2009. [Online]. Available: https://www.usenix.org/conference/usenixsecurity09/technical-sessions/presentation/efficient-data-structures-tamper-evident

[13] B. Laurie, A. Langley, and E. Kasper, “Certificate Transparency,” IETF, RFC 6962, 2013 (Experimental). [Online]. Available: https://www.rfc-editor.org/info/rfc6962

[14] S. Park, M. Specter, N. Narula, and R. L. Rivest, “Going from bad to worse: from Internet voting to blockchain voting,” Journal of Cybersecurity, vol. 7, no. 1, 2021. doi: 10.1093/cybsec/tyaa025.

[15] T. R. Magrino, R. L. Rivest, E. Shen, and D. Wagner, “Computing the Margin of Victory in IRV Elections,” in Proc. 2011 Electronic Voting Technology Workshop / Workshop on Trustworthy Elections (EVT/WOTE '11), San Francisco, CA, 2011. [Online]. Available: https://www.usenix.org/conference/evtwote-11/computing-margin-victory-irv-elections

[16] M. Blom, P. J. Stuckey, and V. Teague, “RAIRE: Risk-Limiting Audits for IRV Elections,” arXiv preprint arXiv:1903.08804 [cs.DS], 2019. [Online]. Available: https://arxiv.org/abs/1903.08804

[17] M. Blom, V. Teague, P. J. Stuckey, and R. Tidhar, “Efficient Computation of Exact IRV Margins,” in Proc. 22nd European Conference on Artificial Intelligence (ECAI 2016), Frontiers in Artificial Intelligence and Applications, 2016. doi: 10.3233/978-1-61499-672-9-480.

[18] A. D. Sarwate, S. Checkoway, and H. Shacham, “Risk-limiting Audits and the Margin of Victory in Nonplurality Elections,” Statistics, Politics and Policy, vol. 4, no. 1, 2013. doi: 10.1515/spp-2012-0003.

[19] S. Aulbach, T. Grust, D. Jacobs, A. Kemper, and J. Rittinger, “Multi-tenant databases for software as a service: schema-mapping techniques,” in Proc. 2008 ACM SIGMOD International Conference on Management of Data, 2008, pp. 1195–1206. doi: 10.1145/1376616.1376736.

[20] D. Jacobs and S. Aulbach, “Ruminations on Multi-Tenant Databases,” in Datenbanksysteme in Business, Technologie und Web (BTW 2007), Lecture Notes in Informatics P-103, Gesellschaft fuer Informatik, Bonn, 2007, pp. 514–521. [Online]. Available: https://dl.gi.de/handle/20.500.12116/31821

[21] P. J. Sadalage and M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley Professional, 2012. [Online]. Available: https://martinfowler.com/books/nosql.html

[22] M. Fowler, “Polyglot Persistence,” martinfowler.com (bliki), 2011. [Online]. Available: https://martinfowler.com/bliki/PolyglotPersistence.html (accessed Oct. 22, 2026).

[23] R. Cattell, “Scalable SQL and NoSQL data stores,” ACM SIGMOD Record, vol. 39, no. 4, pp. 12–27, 2011. doi: 10.1145/1978915.1978919.

[24] M. Stonebraker, “SQL databases v. NoSQL databases,” Communications of the ACM, vol. 53, no. 4, pp. 10–11, 2010. doi: 10.1145/1721654.1721659.

[25] E. Bertino and R. Sandhu, “Database security - concepts, approaches, and challenges,” IEEE Transactions on Dependable and Secure Computing, vol. 2, no. 1, pp. 2–19, 2005. doi: 10.1109/TDSC.2005.9.

[26] P. Antonopoulos et al., “SQL Ledger: Cryptographically Verifiable Data in Azure SQL Database,” in Proc. 2021 International Conference on Management of Data (SIGMOD '21), 2021, pp. 2437–2449. doi: 10.1145/3448016.3457558.

[27] K. E. Pavlou and R. T. Snodgrass, “Forensic analysis of database tampering,” ACM Transactions on Database Systems, vol. 33, no. 4, pp. 1–47, 2008. doi: 10.1145/1412331.1412342.

[28] R. T. Snodgrass, S. S. Yao, and C. Collberg, “Tamper Detection in Audit Logs,” in Proc. 2004 VLDB Conference, 2004, pp. 504–515. doi: 10.1016/b978-012088469-8.50046-2.

[29] “PostgreSQL 18 Documentation, Section 5.5: Constraints,” PostgreSQL Global Development Group, PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/current/ddl-constraints.html (accessed Oct. 22, 2026).

[30] “PostgreSQL 18 Documentation, Chapter 37: Triggers,” PostgreSQL Global Development Group, PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/current/triggers.html (accessed Oct. 22, 2026).

[31] D. Temoshok et al., “Digital Identity Guidelines: Authentication and Authenticator Management,” NIST Special Publication 800-63B-4, National Institute of Standards and Technology, 2025. doi: 10.6028/NIST.SP.800-63b-4.

[32] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache, and O. Ranen, “HOTP: An HMAC-Based One-Time Password Algorithm,” IETF, RFC 4226, 2005 (Informational). [Online]. Available: https://www.rfc-editor.org/info/rfc4226

[33] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, “TOTP: Time-Based One-Time Password Algorithm,” IETF, RFC 6238, 2011 (Informational). [Online]. Available: https://www.rfc-editor.org/info/rfc6238

[34] M. Sönmez Turan, E. Barker, W. Burr, and L. Chen, “Recommendation for Password-Based Key Derivation: Part 1: Storage Applications,” NIST Special Publication 800-132, National Institute of Standards and Technology, 2010. doi: 10.6028/NIST.SP.800-132.

[35] OWASP, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series, OWASP Foundation. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (accessed Oct. 22, 2026).

[36] A. Dmitrienko, C. Liebchen, C. Rossow, and A.-R. Sadeghi, “On the (In)Security of Mobile Two-Factor Authentication,” in Financial Cryptography and Data Security (FC 2014), Lecture Notes in Computer Science, Springer, 2014, pp. 365–383. doi: 10.1007/978-3-662-45472-5_24.

[37] S. L. Garfinkel, “Email-based identification and authentication: an alternative to PKI?,” IEEE Security & Privacy, vol. 1, no. 6, pp. 20–26, 2003. doi: 10.1109/MSECP.2003.1253564.

[38] D. Chaum, “Secret-ballot receipts: true voter-verifiable elections,” IEEE Security & Privacy, vol. 2, no. 1, pp. 38–47, 2004. doi: 10.1109/MSECP.2004.1264852.

[39] J. Benaloh and D. Tuinstra, “Receipt-free secret-ballot elections (extended abstract),” in Proc. 26th Annual ACM Symposium on Theory of Computing (STOC '94), 1994, pp. 544–553. doi: 10.1145/195058.195407.

[40] A. Juels, D. Catalano, and M. Jakobsson, “Coercion-resistant electronic elections,” in Proc. 2005 ACM Workshop on Privacy in the Electronic Society (WPES '05), 2005, pp. 61–70. doi: 10.1145/1102199.1102213.

Appendix A: Schema Definitions

Election database (`setup_db.py`):

```
CREATE TABLE IF NOT EXISTS ORGANIZATION (
    org_id INTEGER PRIMARY KEY AUTOINCREMENT,
    org_name TEXT NOT NULL,
    admin_contact TEXT
)
CREATE TABLE IF NOT EXISTS ELECTION (
    election_id INTEGER PRIMARY KEY AUTOINCREMENT,
    org_id INTEGER,
    status TEXT DEFAULT 'ACTIVE',
    start_time DATETIME,
    end_time DATETIME,
    FOREIGN KEY (org_id) REFERENCES ORGANIZATION(org_id)
)
CREATE TABLE IF NOT EXISTS CANDIDATE (
    candidate_id INTEGER PRIMARY KEY AUTOINCREMENT,
    election_id INTEGER,
    name TEXT NOT NULL,
    platform_statement TEXT,
    FOREIGN KEY (election_id) REFERENCES ELECTION(election_id)
)
CREATE TABLE IF NOT EXISTS VOTER_PARTICIPATION (
    participation_id INTEGER PRIMARY KEY AUTOINCREMENT,
    election_id INTEGER,
    student_reg_no TEXT NOT NULL,
    password_hash TEXT NOT NULL,
    has_voted INTEGER DEFAULT 0 CHECK(has_voted IN (0, 1)),
    FOREIGN KEY (election_id) REFERENCES ELECTION(election_id),
    UNIQUE (student_reg_no, election_id)
)
# Enforcing strict Top-3 ranking constraint at the database layer
CREATE TABLE IF NOT EXISTS BALLOT_RANKS (
    ballot_token TEXT NOT NULL,
    candidate_id INTEGER,
    election_id INTEGER,
    preference_rank INTEGER NOT NULL CHECK (preference_rank BETWEEN 1 AND 3),
    PRIMARY KEY (ballot_token, candidate_id),
    FOREIGN KEY (candidate_id) REFERENCES CANDIDATE(candidate_id),
    FOREIGN KEY (election_id) REFERENCES ELECTION(election_id)
)
# 2. Native SQL Trigger: Hard block against duplicate voting
CREATE TRIGGER IF NOT EXISTS trg_block_duplicate_vote
BEFORE UPDATE OF has_voted ON VOTER_PARTICIPATION
FOR EACH ROW
WHEN OLD.has_voted = 1 AND NEW.has_voted = 1
BEGIN
    SELECT RAISE(ABORT, 'DBMS Trigger Violation: Voter has already submitted a ballot.');
END;
# 3. Native SQL Views: Analytics for DBA Review
CREATE VIEW IF NOT EXISTS VIEW_ELECTION_TURNOUT AS
SELECT
    e.election_id,
    o.org_name,
    COUNT(v.participation_id) AS total_roster,
    SUM(v.has_voted) AS total_voted,
    ROUND((CAST(SUM(v.has_voted) AS FLOAT) / COUNT(v.participation_id)) * 100, 2) AS turnout_percentage
FROM ELECTION e
JOIN ORGANIZATION o ON e.org_id = o.org_id
LEFT JOIN VOTER_PARTICIPATION v ON e.election_id = v.election_id
GROUP BY e.election_id;
CREATE VIEW IF NOT EXISTS VIEW_FIRST_PREFERENCES AS
SELECT
    c.candidate_id,
    c.name AS candidate_name,
    COUNT(b.ballot_token) AS first_preference_votes
FROM CANDIDATE c
LEFT JOIN BALLOT_RANKS b ON c.candidate_id = b.candidate_id AND b.preference_rank = 1
GROUP BY c.candidate_id, c.name;
```

Verification database (`verification/otp.py`):

```
CREATE TABLE IF NOT EXISTS VOTER_CONTACT (
            election_id INTEGER NOT NULL,
            reg_no TEXT NOT NULL,
            email TEXT NOT NULL,
            PRIMARY KEY (election_id, reg_no));
        CREATE TABLE IF NOT EXISTS OTP_CHALLENGE (
            challenge_id TEXT PRIMARY KEY,
            election_id INTEGER NOT NULL,
            reg_no TEXT NOT NULL,
            code_hash TEXT NOT NULL,
            created_at REAL NOT NULL,
            expires_at REAL NOT NULL,
            last_sent_at REAL NOT NULL,
            sends INTEGER NOT NULL DEFAULT 1,
            attempts INTEGER NOT NULL DEFAULT 0,
            state TEXT NOT NULL DEFAULT 'open' CHECK (state IN ('open','used','locked')));
        CREATE TABLE IF NOT EXISTS MAIL_LOG (
            log_id INTEGER PRIMARY KEY AUTOINCREMENT,
            sent_at REAL NOT NULL,
            election_id INTEGER NOT NULL,
            reg_no TEXT NOT NULL,
            email TEXT NOT NULL,
            status TEXT NOT NULL);
```

Appendix B: REST Interface

Method and path	Who	Purpose
POST `/api/voter/login`	Voter	Check password; return a session, or an OTP challenge when verification is on
POST `/api/voter/otp/verify`	Voter	Exchange challenge and code for a voting session
POST `/api/voter/otp/resend`	Voter	Send a fresh code (rate-limited)
GET, POST `/api/voter/ballot`	Voter	Fetch candidates; cast the ranked ballot
GET `/api/receipts/{token}`	Public	Check that a ballot is sealed and counted
GET `/api/elections`, `/api/config`, `/api/health`	Public	Open elections; public URL; ledger backend and sealing mode
POST `/api/admin/login`	Admin	Administrator sign-in
`/api/admin/elections` (GET, POST), `.../{id}`, `.../status`	Admin	Create, inspect and advance elections
`.../candidates`, `.../candidates/seed`	Admin	Manage candidates (Nomination only)
`.../voters`, `.../voters/bulk`	Admin	Manage the roster
`.../contacts`, `.../contacts/fill`	Admin	Voter emails and bulk plus-address fill
GET `/api/admin/otp/status`, `/api/admin/otp/outbox`	Admin	Verification settings; recent codes
GET `.../integrity`, `.../tally`, `/api/admin/ledger`	Admin	Sweep, instant-runoff count, ledger and chain check
`.../studio/*`	Admin	Simulated rogue-administrator edits for the tamper demonstration

Table 30: Principal endpoints.

Appendix C: Reproducing the Experiments

From the project directory, with Python 3.12 and the requirements installed:

```
python experiments/run_experiments.py patched_r1     # final design (keyed chain on)
python experiments/run_experiments.py nokey          # final design, key off
python experiments/summarize.py                      # medians, summary.json, charts
python experiments/bench_mongo.py                    # Atlas append latency (needs MONGO_URI)
```

The baseline was produced by pointing the same script at an earlier checkout with `EXP_SRC=<path>`. Each run writes `experiments/results/<tag>.json`. Runs use temporary copies of the application and never read or write the real database, ledger or `.env`.

Appendix E: Demo Walkthrough and Workflow

Public demo link: `https://polyvote-bacse202.onrender.com` (free hosting: the first visit after idle can take about a minute). **Source code:** https://github.com/Tharakeshtj/polyvote-election-platform. The `/demo` page inside the application lists every page.

E.1 Workflow of one election

Step	Who	What happens	Where
1	Officer	Creates the organisation and election	Admin, New election
2	Officer	Adds candidates (only before voting opens)	Election, Candidates
3	Officer	Registers voters and their email addresses	Election, Voters
4	Officer	Opens voting	Election, Overview
5	Voter	Opens the vote page (or scans the QR code), signs in with registration number and password	Vote page
6	Voter	Enters the 6-digit code received by email	Vote page
7	Voter	Ranks three candidates, reviews and casts; keeps the receipt token	Vote page
8	Anyone	Checks that the receipt is sealed and counted	Check receipt
9	Officer	Closes voting, runs the integrity sweep, counts the ballots	Election, Results
10	Officer	Tamper test: edits or deletes a ballot in DB Studio, runs the sweep again	Election, DB Studio and Results

Table 31: Workflow of one election from set-up to audit.

E.2 Three-minute tour of the public demo

- Open the demo link and read the "Demo mode" banner; click **Demo guide** to see every page.
- Click **Vote** and sign in as `25BCE0025` with password `vote123` (voters `25BCE0019` to `25BCE0030` have not voted).
- Read the 6-digit code from the blue "Demo mode" box, type it in, rank three candidates, review and cast. Copy the receipt token.
- Open **Check receipt**, paste the token: the ballot is sealed in the ledger and counted.
- Open **Admin** (`admin` / `admin123` on the public demo only). Go to Election 1, **Results**, and run the sweep: it passes.

- In **DB Studio**, delete a ballot, return to Results and run the sweep again: the deleted ballot is named. Open **Audit ledger**: the chain is intact. Open **Mail outbox**: every code issued is listed.

E.3 Class demonstration with real email codes

- Start the sample build with `START_DEMO.bat` and connect a Gmail App Password once with `SET_GMAIL.bat`.
- Create a new election, add five candidates, bulk-register five voters, and set their emails; open voting. Add two more voters by hand, each with their own email.
- Run the ballot simulation for the first four voters, then vote for the remaining three from a phone by scanning the QR code: each voter signs in, receives their own code by email, and casts.
- Close the election, run the sweep and the count, then edit one block in MongoDB Atlas and reload the Audit ledger to show the red "tampered" banner.

Appendix D: Detail of Tampering Scenarios

ID	Scenario	Final design: evidence
C1	Swap the 1st and 2nd choice on one ballot	flags: sweep; counts {'voted_flags': 40, 'ballots': 40, 'receipts': 40}
C2	Replace the first-choice candidate on one ballot	flags: sweep; counts {'voted_flags': 40, 'ballots': 40, 'receipts': 40}
C3	Delete one ballot	flags: sweep, counters; counts {'voted_flags': 40, 'ballots': 39, 'receipts': 40}
C4	Delete a ballot and clear a voted flag to hide the gap	flags: sweep, counters; counts {'voted_flags': 39, 'ballots': 39, 'receipts': 40}
C5	Inject a ballot with no receipt	flags: sweep, counters; counts {'voted_flags': 40, 'ballots': 41, 'receipts': 40}
C6	Ballot stuffing: inject ballot, forge receipt, set a voted flag	flags: chain; counts {'voted_flags': 41, 'ballots': 41, 'receipts': 41}
C7	Edit a ledger block without repairing the chain	flags: sweep, chain; counts {'voted_flags': 40, 'ballots': 40, 'receipts': 40}
C8	Delete a block from the middle of the ledger	flags: sweep, chain, counters; counts {'voted_flags': 40, 'ballots': 40, 'receipts': 39}
C9	Truncate the last block of the ledger	flags: sweep, counters; counts {'voted_flags': 40, 'ballots': 40, 'receipts': 39}
C10	Change a ballot and rebuild the whole chain consistently (attacker has no secret key)	flags: chain; counts {'voted_flags': 40, 'ballots': 40, 'receipts': 40}
C11	Swap two candidates' names after voting	flags: sweep; counts {'voted_flags': 40, 'ballots': 40, 'receipts': 40}
C12	Reset a voted flag, then let that voter cast a second ballot	flags: sweep, counters; counts {'voted_flags': 40, 'ballots': 41, 'receipts': 41}
C13	Add a ghost voter to the roster and vote as them through the API	flags: none; counts {'voted_flags': 41, 'ballots': 41, 'receipts': 41}

Table 32: Evidence recorded for each scenario under the final design.

"flags" lists which checks fired: `sweep`, `chain`, or `counters`. "counts" are voted flags, ballots and receipts as seen by the administrator interface.